

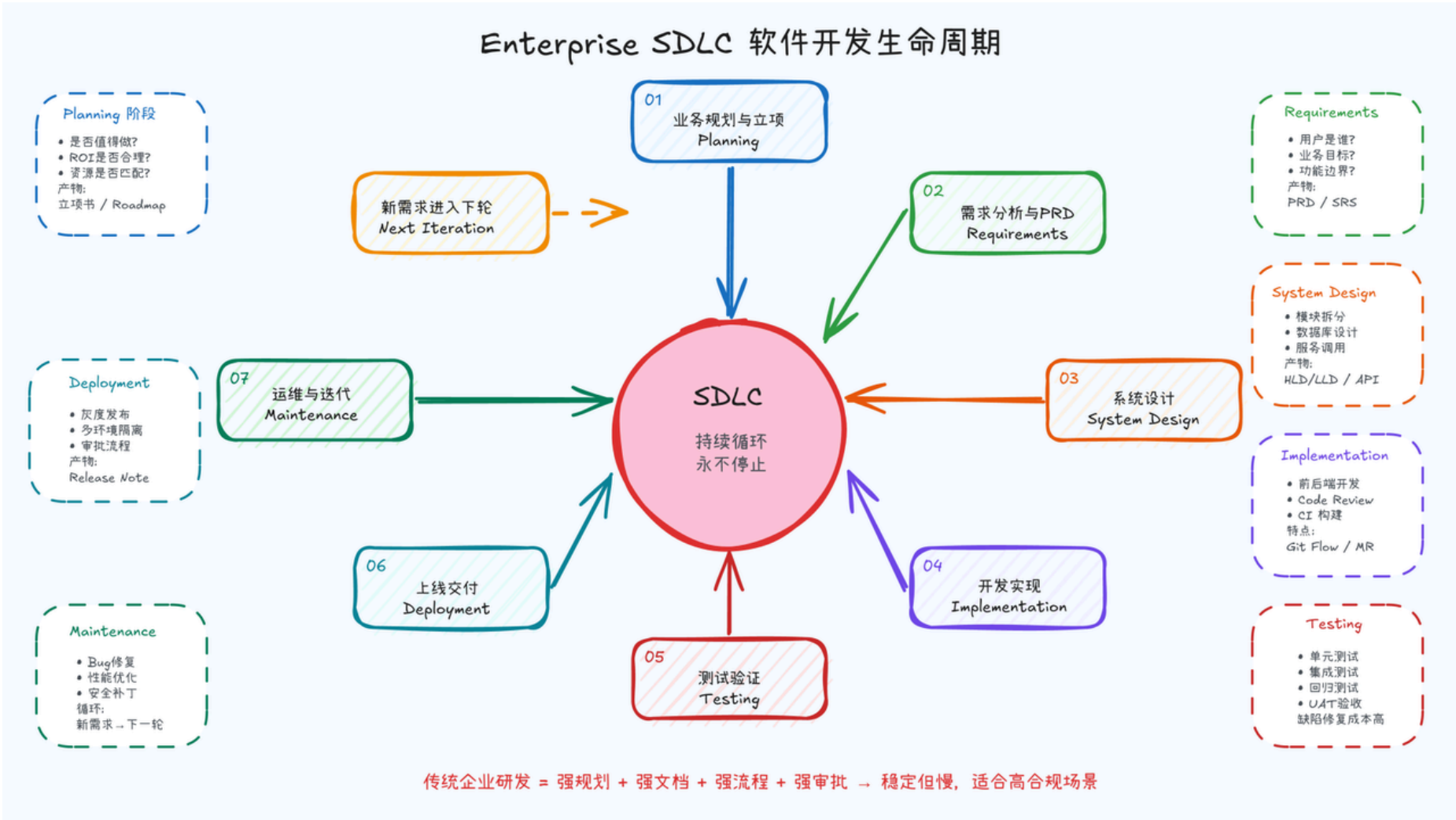
Cursor 编程深度实践-智能问数平台

本课程基于 Cursor 开发**企业智能问数平台**的实战流程，遵循规划立项、需求分析、系统设计等标准项目生命周期，全程运用 Cursor 及大模型提效。重点阐述 Cursor 的 **MCP 接入**、**Rules 规则配置**、**AI 任务管理**等高阶功能：通过 MCP 集成 Linear 和 LangChain MCP 服务，实现项目管理与框架文档的便捷调用；利用 Rules 四种注入模式构建自动化研发 workflow；打通 Cursor 与 Linear，实现开发任务的 AI 管理和状态实时同步。项目按阶段拆开发任务，依托 Cursor 完成前后端开发、联调与优化，最终实现自然语言交互的智能数据分析平台的开发落地。

1、概述

本章将带领大家基于 Cursor 开发**企业智能问数平台**项目案例。在完整的从零到一实战开发过程中，我们将重点演示 Cursor 的多项高阶功能，包括 MCP 接入、Rules 规则配置、AI 驱动的项目进度管理等，帮助大家真正掌握 Cursor 在实际工程中的高效使用方法，实现从入门到熟练应用的全面提升。

如下是我们开发一个完整项目的生命周期：



- **规划立项（Planning）**：从商业视角论证可行性，评估 ROI 与资源匹配度，产出立项书与 Roadmap，锚定项目方向。
- **需求分析（Requirements）**：明确用户、业务目标与功能边界，输出 PRD/SRS，将商业诉求转化为可执行的产品需求。
- **系统设计（System Design）**：完成模块拆分、数据库与服务设计，产出 HLD/LLD/API 文档，为开发提供技术蓝图。
- **开发实现（Implementation）**：前后端编码、代码评审与 CI 构建，通过 Git Flow 等流程保障代码质量与协作效率。
- **测试验证（Testing）**：覆盖单元、集成、回归及 UAT 验收，高成本修复缺陷，保障上线质量。
- **上线交付（Deployment）**：通过灰度发布、多环境隔离与审批流程，平稳交付版本，产出 Release Note。
- **运维迭代（Maintenance）**：负责 Bug 修复、性能优化与安全补丁，新需求再次进入下一轮循环，实现持续演进。

我们基于 Cursor 等 AI 编程工具编程也是按照这个流程来的！

2、项目立项

我们先推进项目的**第一步：项目立项**。立项的核心目标，是**明确项目定位、业务价值与核心边界**，把“要做什么、解决什么问题、达成什么效果”梳理清晰。

本次立项的目标明确且聚焦：**建设一套智能数据分析平台**。平台以**自然语言交互**为入口，用户无需编写代码或复杂操作，只需输入业务问题或分析需求，系统即可自动完成数据解析、逻辑计算、统计分析，并输出专业、直观的数据可视化结果与分析结论，实现“**自然语言提问 — 自动数据分析 — 智能可视化呈现**”的一站式体验。

3、需求分析

在本阶段，我们可借助**ChatGPT、Gemini**等主流大语言模型，将项目需求与业务场景输入模型，利用其**深度检索（Deep Search）**能力，自动完成行业方案调研、技术路径分析、同类产品对比等工作。基于模型输出的结构化调研报告，我们进一步**明确项目整体技术方案、完成核心技术栈选型**，为后续研发落地提供可靠依据。

登录 ChatGPT，我们给 ChatGPT 如下的提示词：

代码块

- 1 我当前的需求如下：
- 2 1.接入大模型
- 3 2.能够操作数据库，基于用户的自然语言查询数据库返回最终的结果
- 4 3.需要在前端实时的渲染查询出来的可视化图表
- 5 实现“自然语言提问 – 自动数据分析 – 智能可视化呈现”的一站式体验。
- 6
- 7 我需要你基于我的这个需求，详细的进行底层架构的设计，主要包含如下几点：
- 8 1.如果要实现这个系统，我可以采用大模型领域的哪些核心技术
- 9 2.目前有没有一些比较热门，且企业常用的Agent开发框架比较适合完成这个需求，
- 10 3.有没有一些开源项目可以推荐
- 11 4.如果要做这个系统的话，前后端的技术栈建议如何设计
- 12
- 13 接入的大模型我想用Qwen在线模型，关系数据库用SQLite数据库，然后项目能直接在本地运行即可。

我们先从哪里开始呢？

我需要你基于我的这个需求，详细的进行底层架构的设计，主要包含如下几点:

1.如果要实现这个系统，我可以采用大模型领域的哪些核心技术

2.目前有没有一些比较热门，且企业常用的Agent开发框架比较适合完成这个需求，

3.有没有一些开源项目可以推荐

4.如果要做这个系统的话，前后端的技术栈建议如何设计

接入的大模型我想用Qwen在线模型，关系数据库用SQLite数据库，然后项目能直接在本地运行即可。|

+ 🔍 深度研究 ▾ 🧩 应用 ▾ 🌐 站点 ▾



推荐 报告

- 🔪 分析技能需求 基于招聘数据、薪酬水平与技术发展趋势，哪些技能的需求增长最快，哪些技能正在贬值？揭示哪些
- 🔪 评估食品负担能力 分析各区域的食物价格、营养状况及家庭支出。识别食物成本上升最快的地区，以及哪些饮食方
- 🔪 追踪气候影响 分析近期的气候、能源及环境数据，识别受极端天气、能源转型和监管政策变化影响最显著的地区与
- 🔪 比较信贷方案 分析信用卡、个人贷款及抵押贷款的利率、费用与借款人还款情况。识别哪些信贷类型正变得更为具

最终 ChatGPT 会返回来一个调研报告！

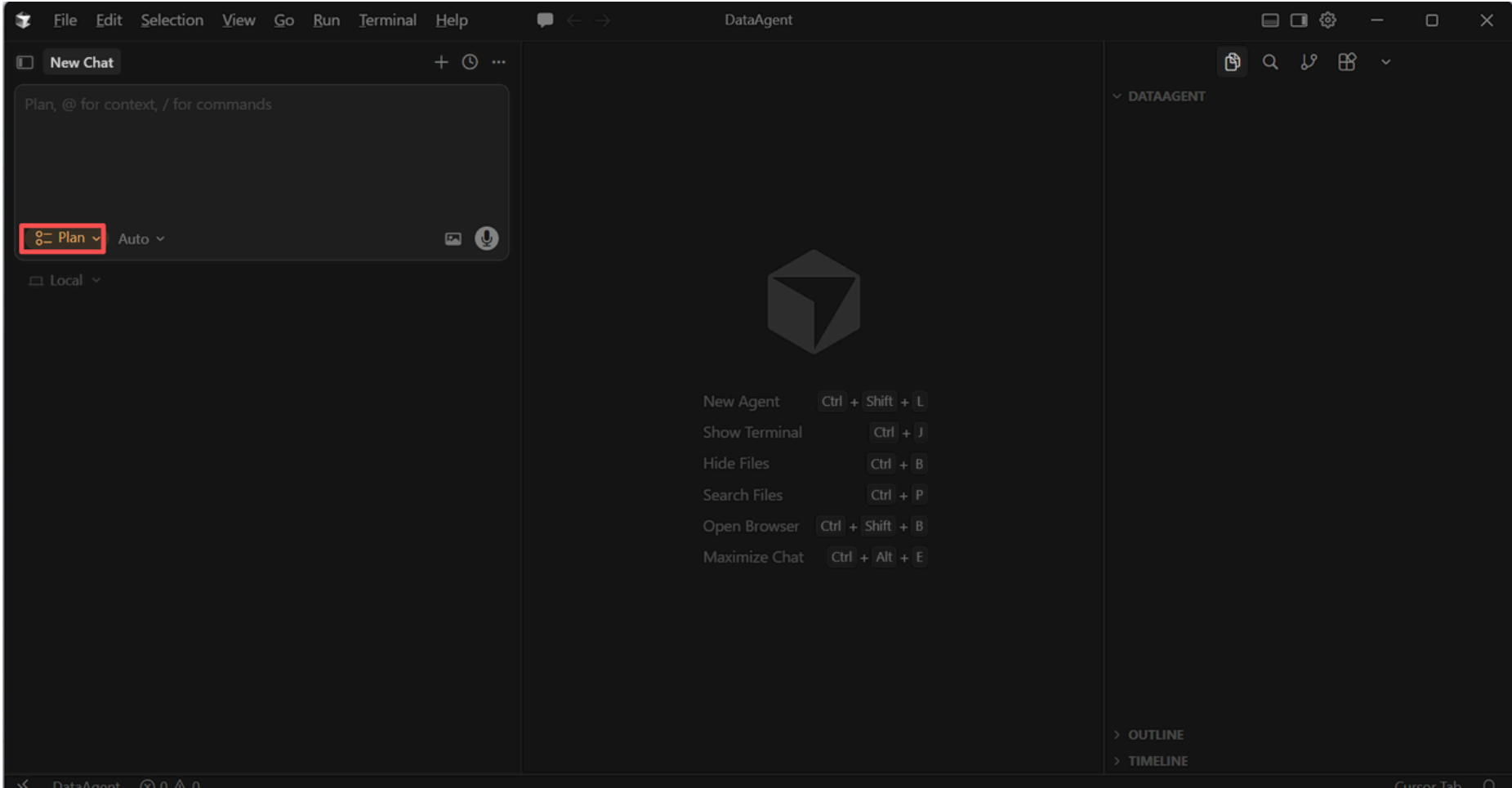
4、架构设计

结合 CahtGPT 输出的需求分析与技术调研报告，我们综合评估并确定本项目整体技术架构如下：

- 前端交互层：采用 **ReAct** 框架实现用户界面与交互逻辑；
- 后端核心层：基于 **Qwen3** 大模型提供语言理解与推理能力，搭配 **LangChain** 构建智能分析流程，并通过 **SQL Database** 完成数据存储与查询。

接下来进入 Cursor，使用 plan 模式，完成系统设计。

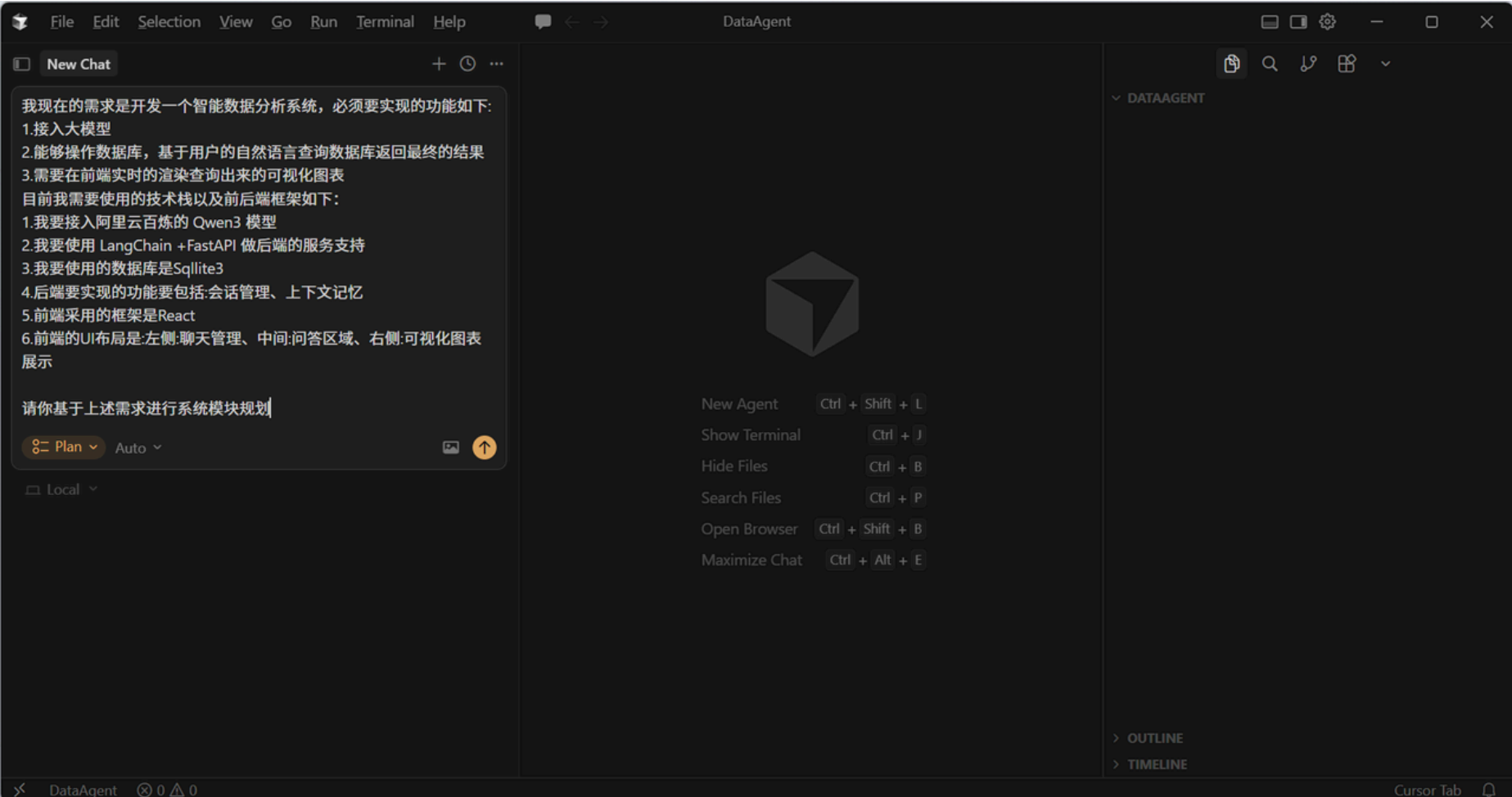
打开 Cursor，切换为 plan 模式。



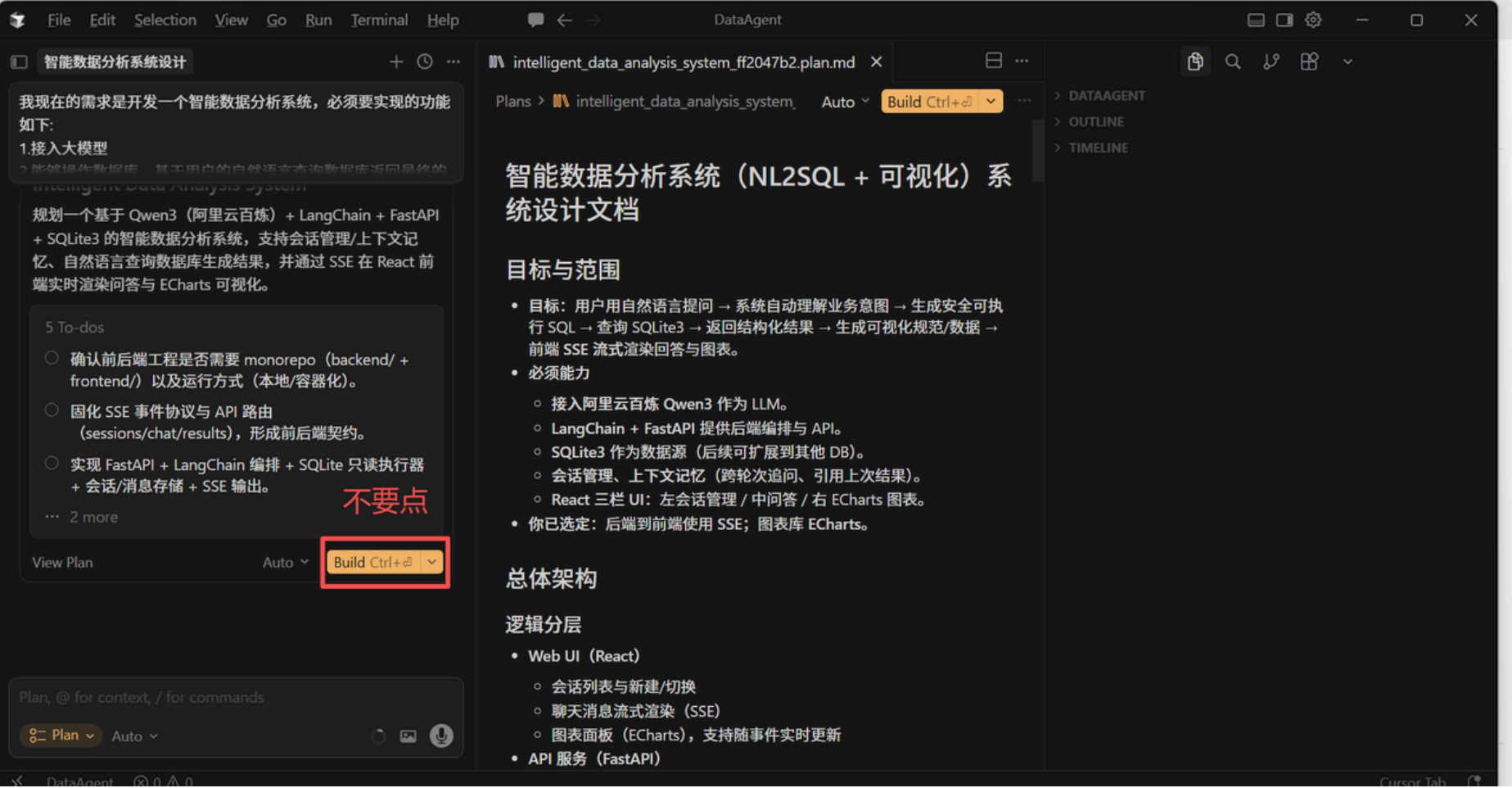
参考提示词：

代码块

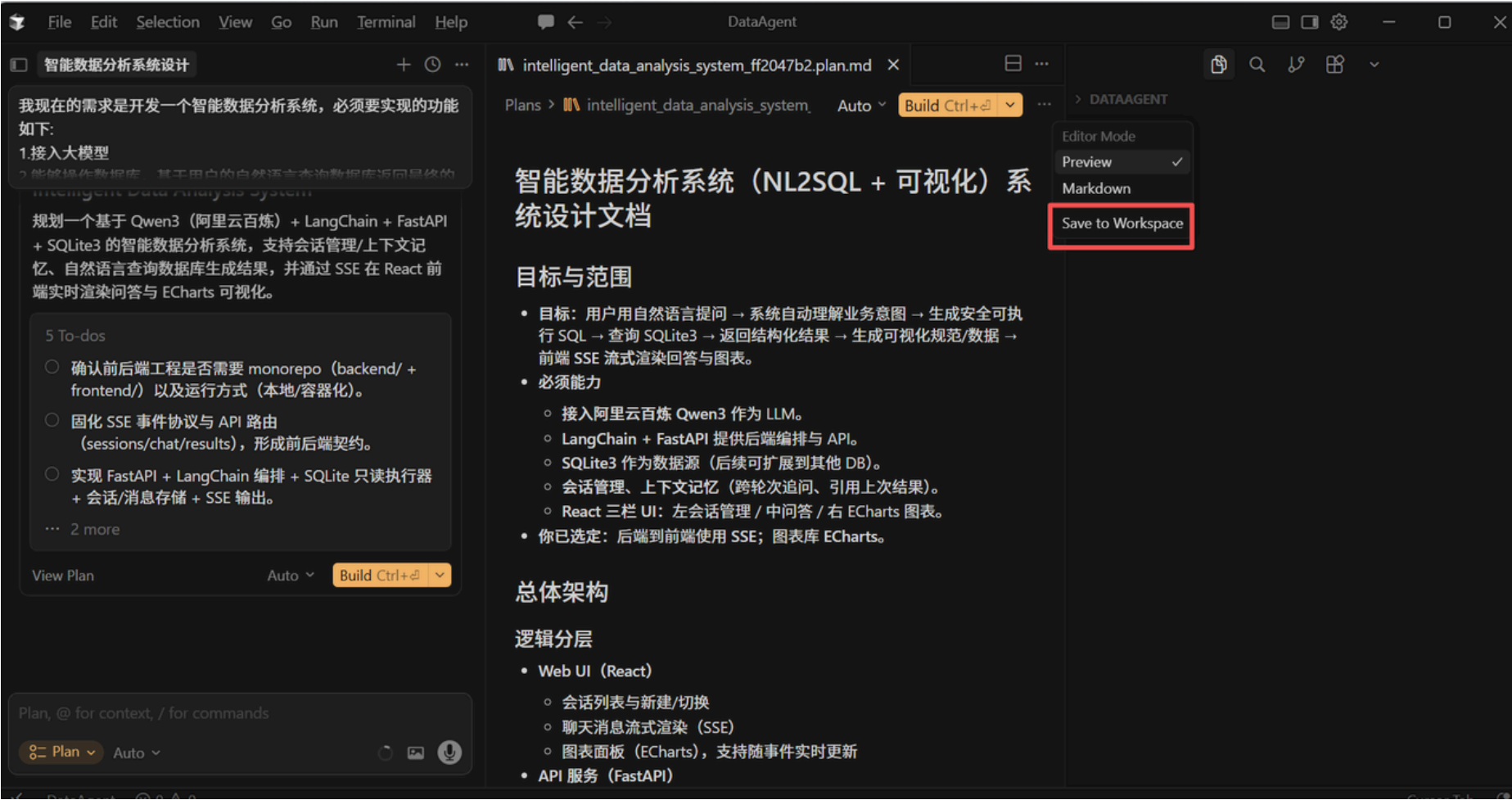
- 1 我当前的需求是开发一个智能数据分析系统，必须要实现的功能如下：
- 2 1.接入大模型(Qwen3)
- 3 2.能够操作数据库，基于用户的自然语言生成SQL语句，然后自动查询数据库返回最终的结果
- 4 3.需要在前端实时的渲染查询出来的可视化图表来展示结果
- 5
- 6 目前我需要使用到的技术栈以及前后端框架如下：
- 7 1.我要接入阿里云百炼的 Qwen3 模型
- 8 2.我要使用 LangChain +FastAPI 做后端的服务支持
- 9 3.我要使用的数据库是 Sqlite
- 10 4.后端要实现的功能要包括:会话管理、上下文记忆
- 11 5.前端采用的框架是: React
- 12 6.前端的UI布局是:左侧:聊天管理、中间:问答区域、右侧:可视化图表展示
- 13
- 14 请你基于上述需求进行系统模块规划



如果出现需要交互，你根据实际情况选择即可。执行完成后，会生成对应的系统设计文档：



这个文档里面如果有不合适的可以改，也可以多次跟Cursor交互，最终确认后，我们保存此文件。



此时在 Cursor 中会生成多条 TODO 开发任务。若直接点击 **Build** 按钮，Cursor 会按照 TODO 列表**自动批量执行开发**，一次性完成大量代码生成与修改。这种方式虽然高效，但一次性处理内容过多，**极易引发逻辑冲突、代码异常或整体功能不可用**等问题。

因此，**建议暂不点击 Build 按钮**，而是由团队根据项目进度与开发节奏，**自主拆解、规划与分步执行**开发任务，确保代码质量与系统稳定性。

5、功能拆分

基于上面的文档，让 Cursor 进一步拆分 Phase。

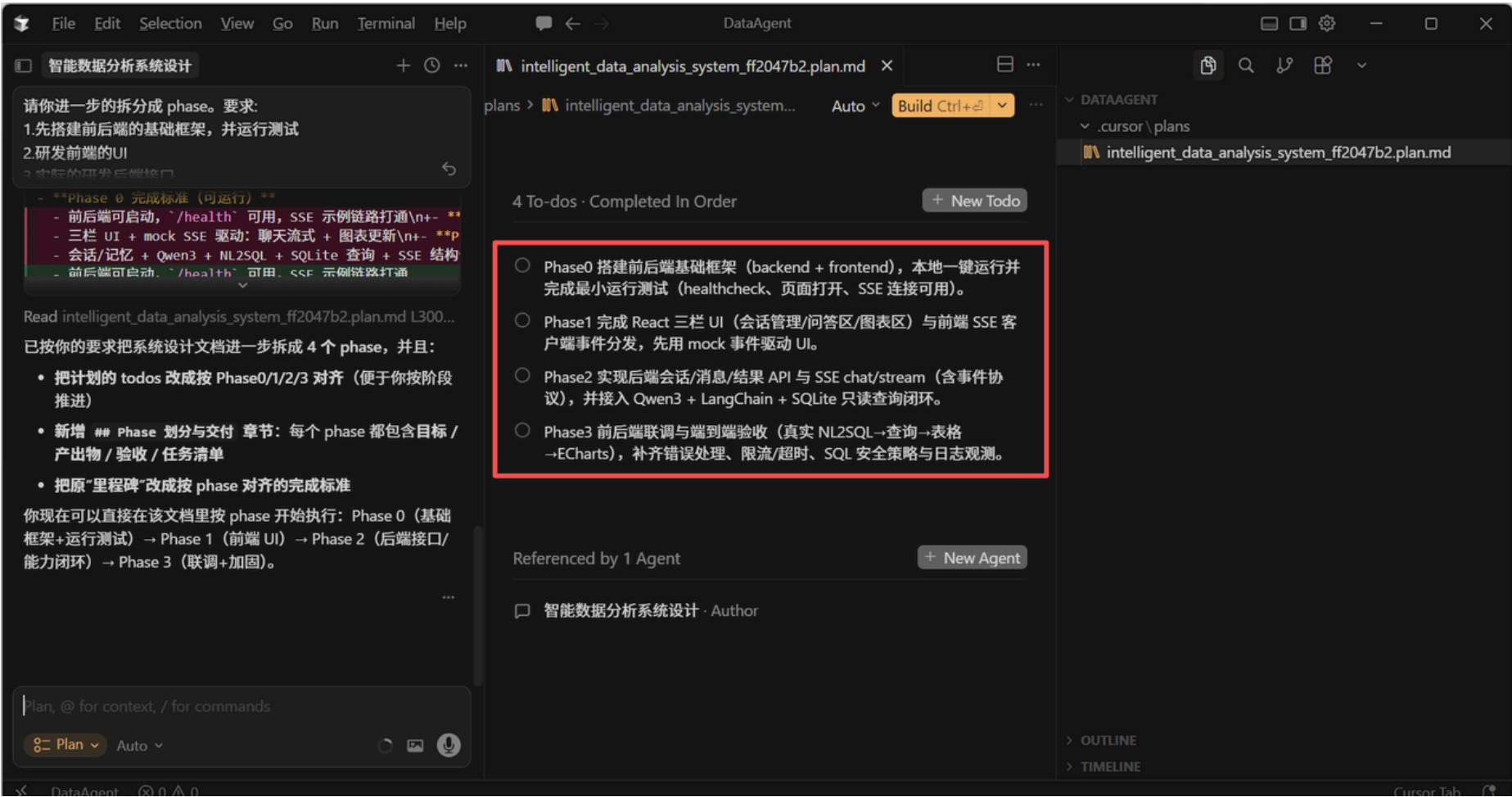
参考提示词

代码块

- 1 请你进一步的拆分成 phase。要求：
- 2 phase0:先搭建前后端的基础框架，并运行测试
- 3 phase1：开发出前端的UI，并且让我登录测试
- 4 phase2：开发后端接口
- 5 phase3：完成前后端联调

这一开发流程既符合我个人的开发习惯，也是业内多数程序员在实际项目中普遍采用的稳健思路：**先完成项目整体框架搭建，再逐步实现前端界面与交互逻辑，随后进行前后端接口对接与数据联调，最后开展全链路联调测试**。按照这种分层推进、逐步集成的方式开展研发工作，能够有效降低开发风险，避免因模块耦合过紧导致的问题，整体开发过程通常更加稳定、可控。

执行完成后，我们的系统设计文档里面就有了如下四个 TODO。



截至目前，项目整体任务已完成初步拆分，各位可结合自身开发节奏与项目实际进度，对任务进行**进一步细化拆解**，例如将原规划的 4 个核心阶段，细化为 8~10 个更具可执行性的 TODO 事项。

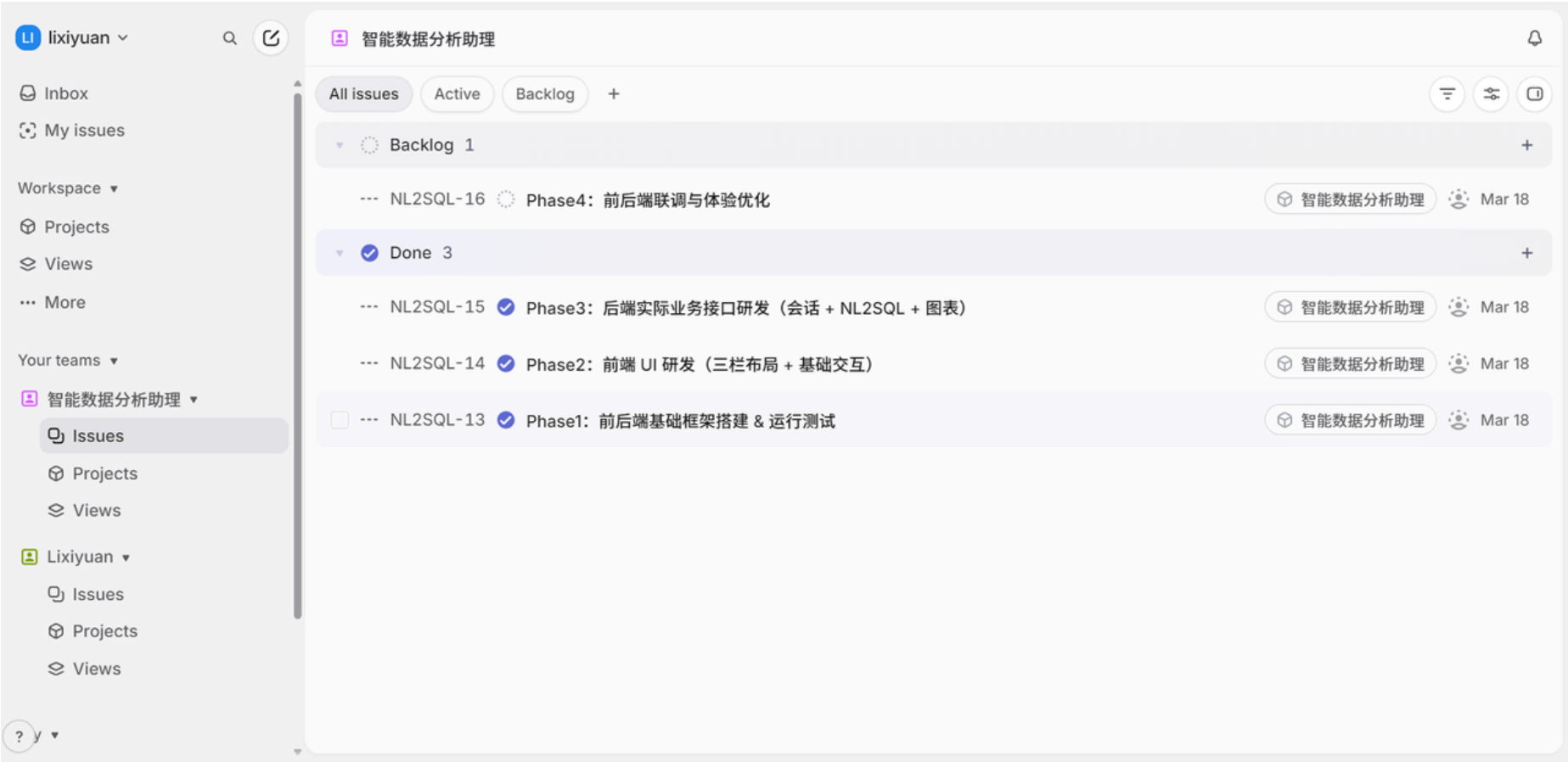
在正式进入编码开发环节之前，我们先暂停开发动作，重点讲解在 **Cursor** 编辑器中，如何高效、规范地开展**团队协作开发**，保障团队协同效率与代码质量。

6、任务管理

1.Linear 平台介绍

针对大型项目的研发工作，高效的团队协作与规范化的项目管理是保障开发进度、提升协作效率的关键，今天为大家推荐一款专业的项目管理工具 **Linear**。该工具的官方登录地址为：<https://linear.app/login>，大家可先前往该地址完成账号注册操作。Linear 是一款免费的在线项目管理平台，能够适配团队协作开发的各类管理需求，助力大型项目的任务拆解、进度追踪与团队协同。

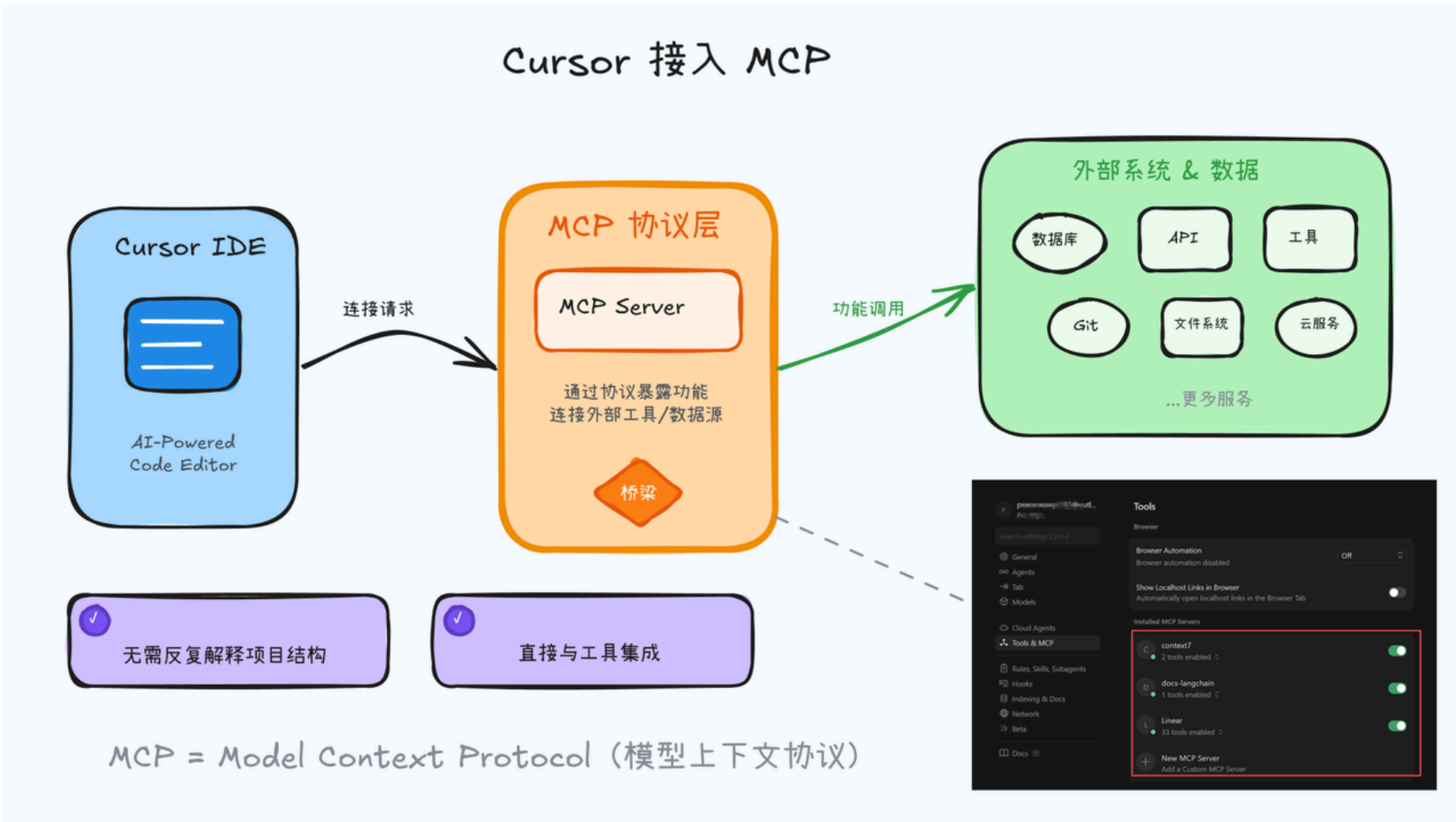
效果界面如下：



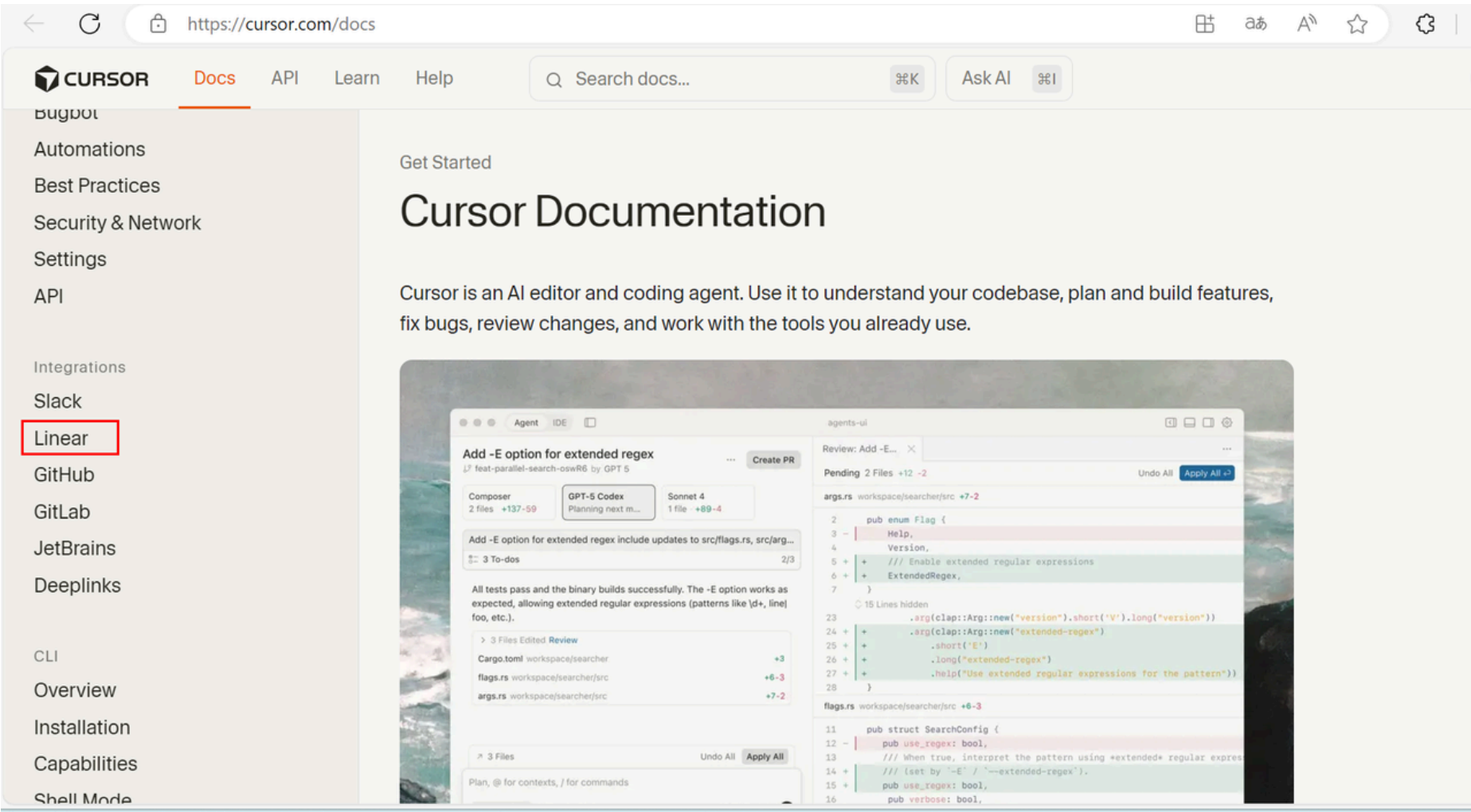
接下来我们需要 Cursor 和 Linear 打通。

2.打通 Cursor 和 Linear

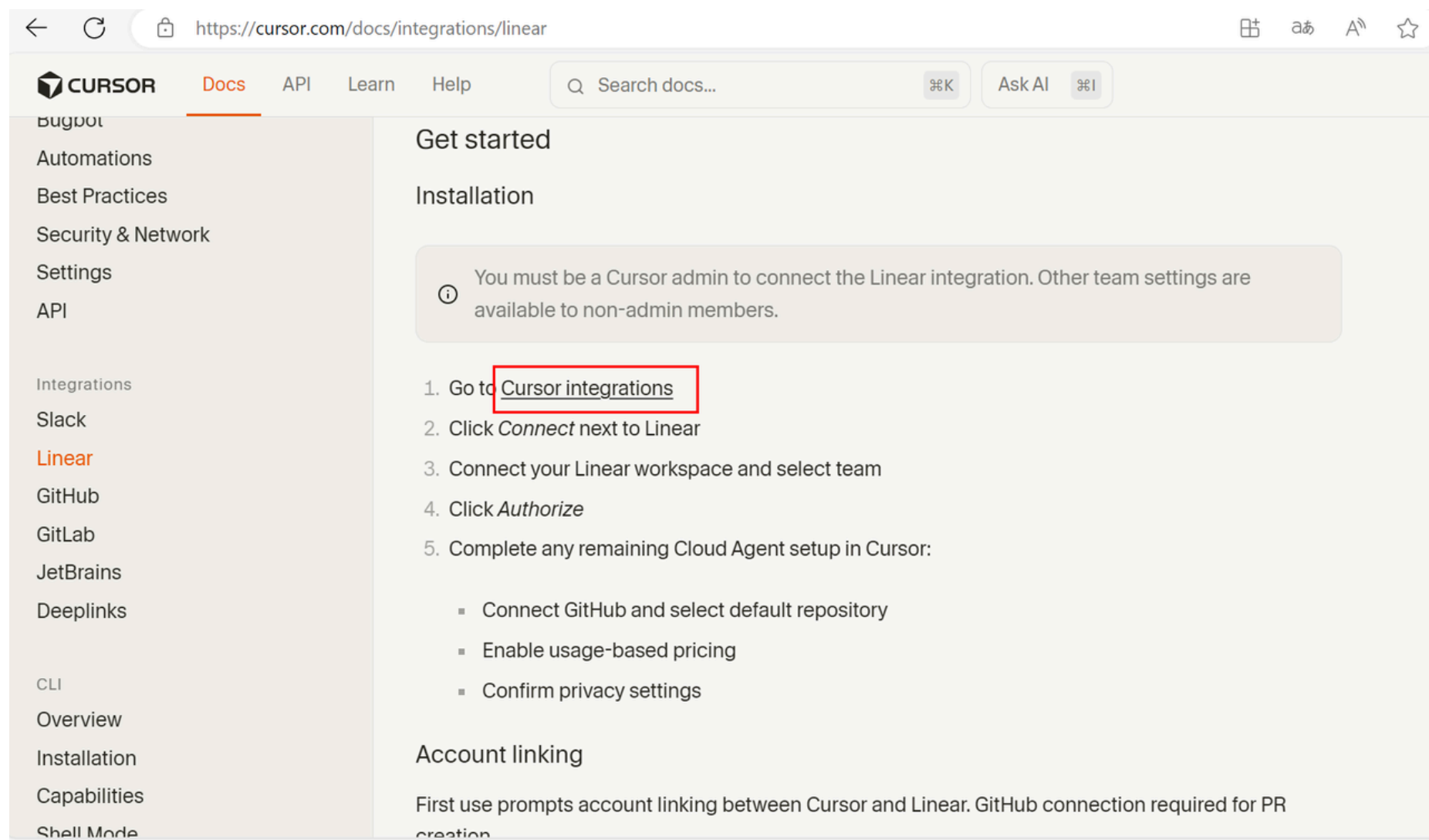
Cursor 是直接连接不上去的，所以要借助于 MCP 的工具。



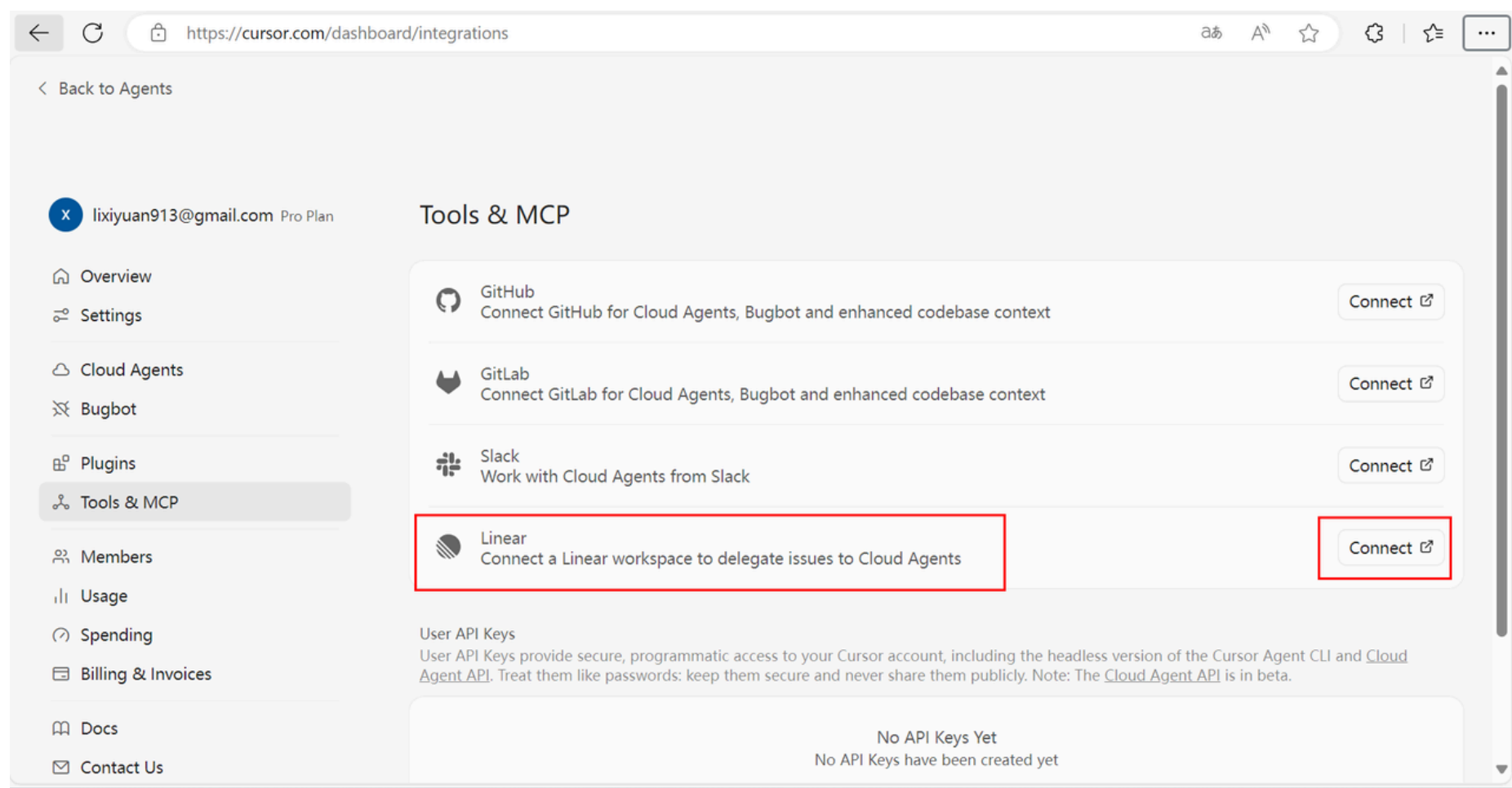
到 Cursor 的官方文档可以看到如何集成 Linear 管理工具。



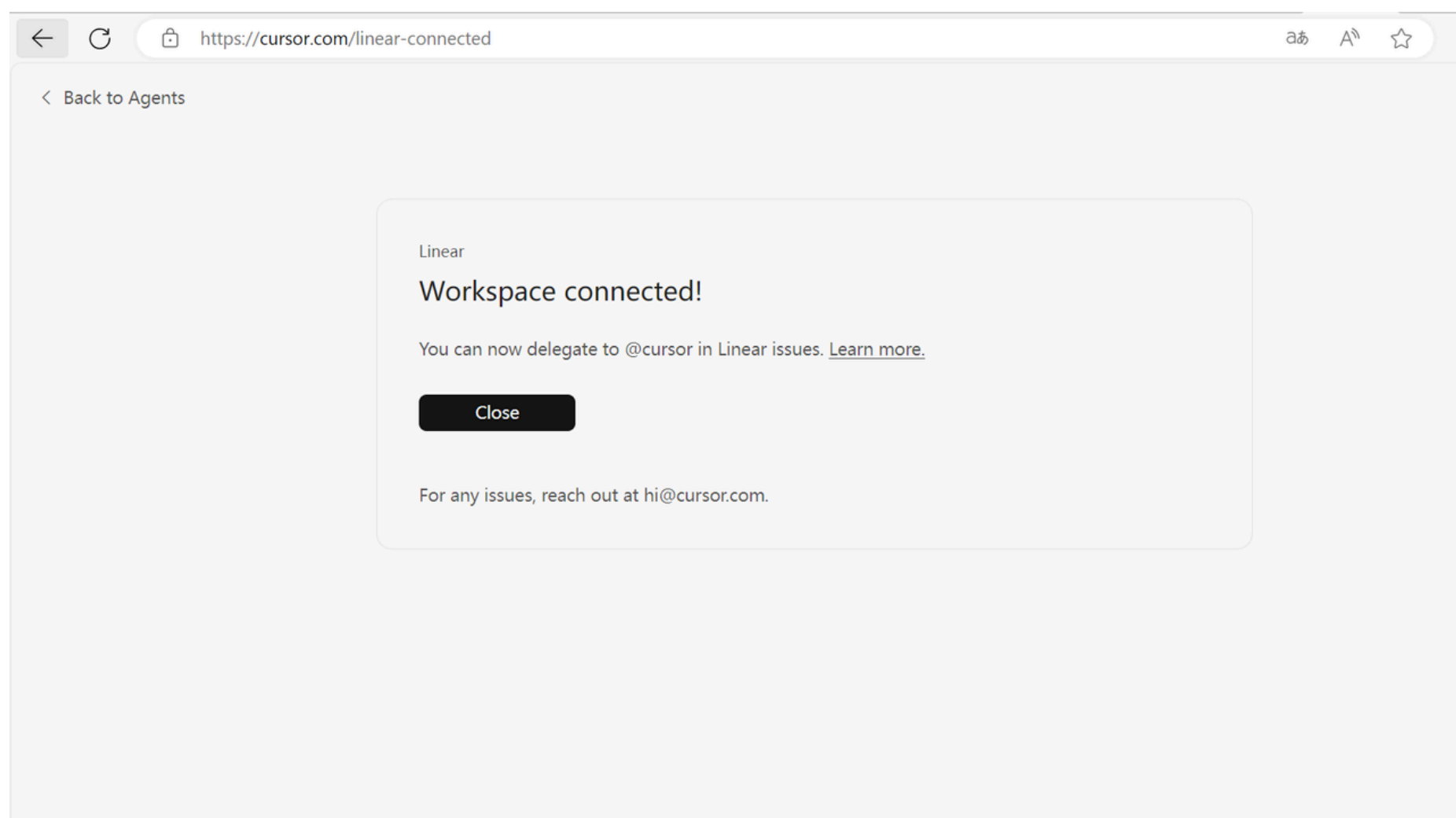
点击图中按钮：



点击图中按钮：



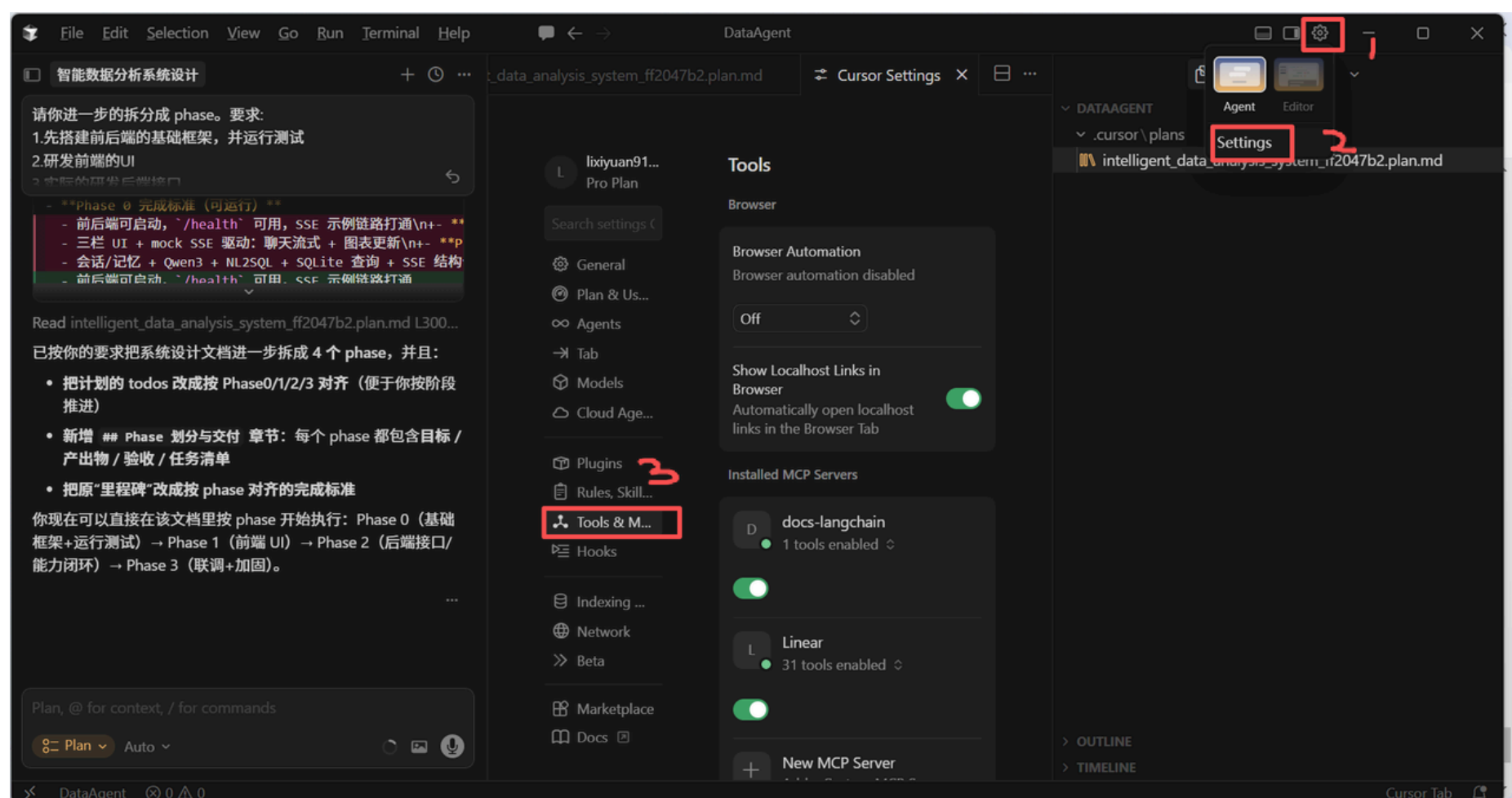
连接成功！



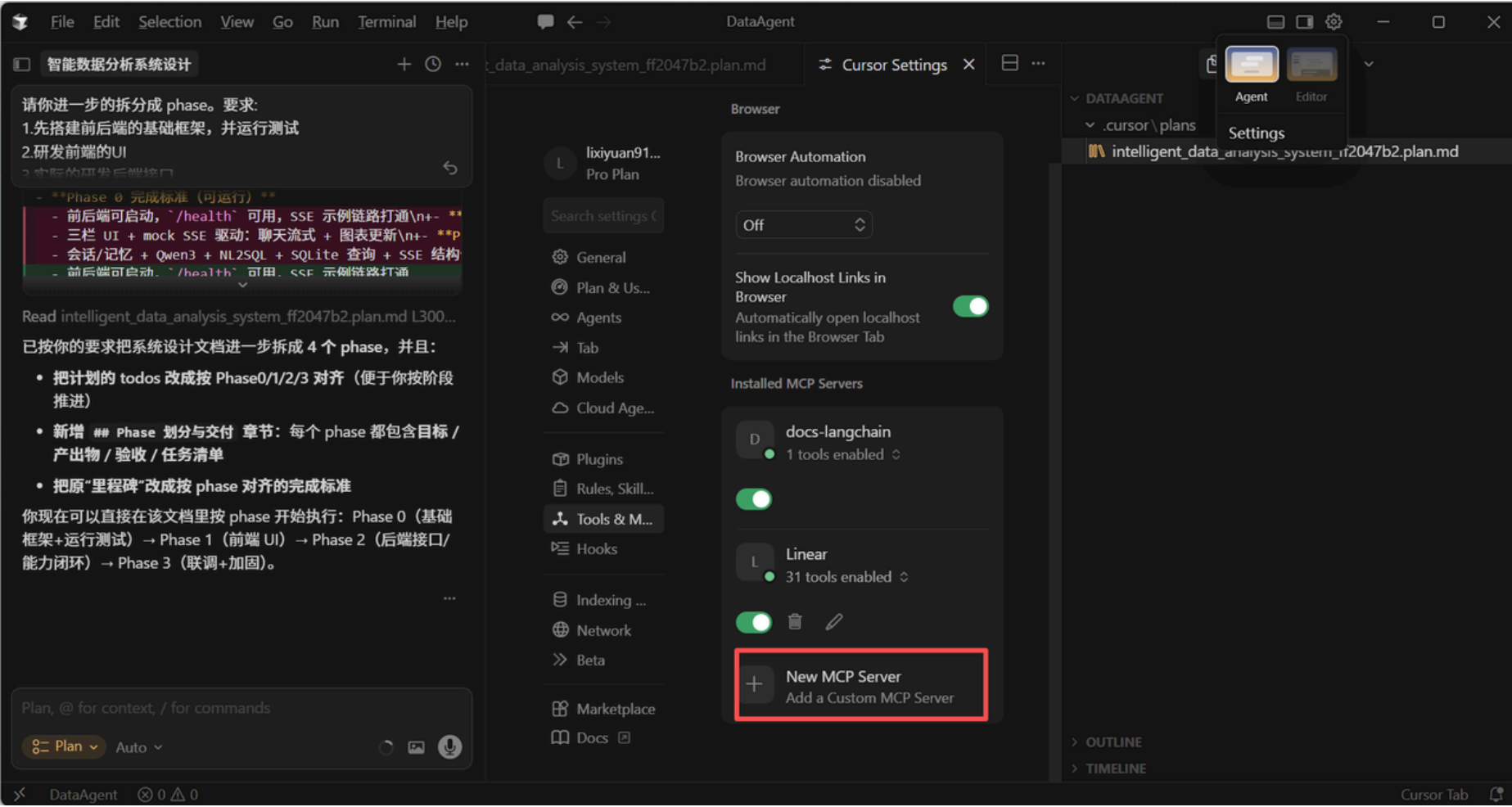
3.Cursor 接入 Linear

在完成 Cursor 与 Linear 的集成后，我们需要在 Cursor 中接入 **Linear** 的 **MCP 服务**。完成配置后，即可直接在 Cursor 编辑器内通过指令对 Linear 进行任务管理、状态更新等操作，实现**编码环境与项目管理平台的无缝打通**，提升团队协作与开发效率。

点工具按钮，接入 MCP：

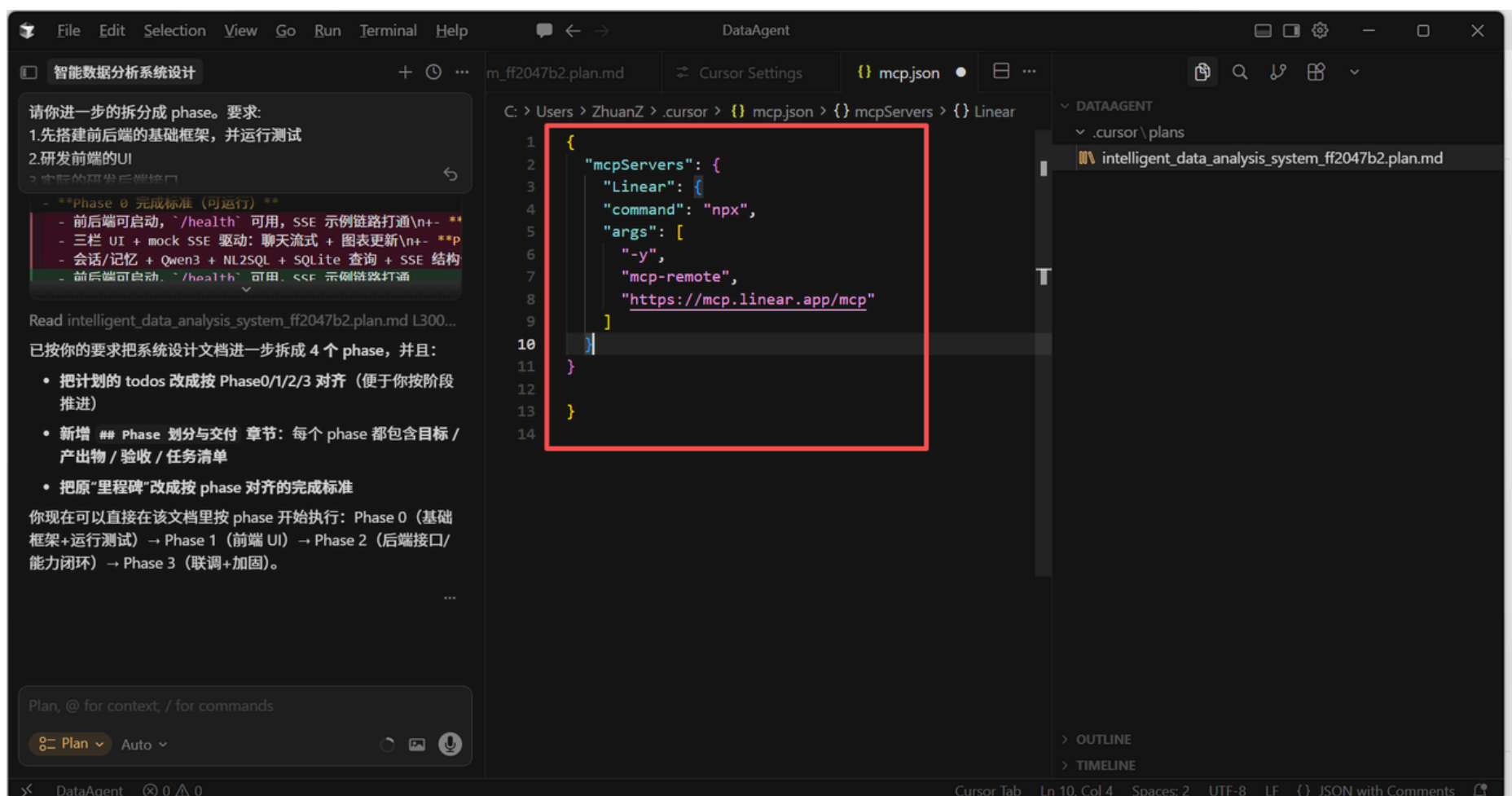


点击新增 MCP 功能按钮：

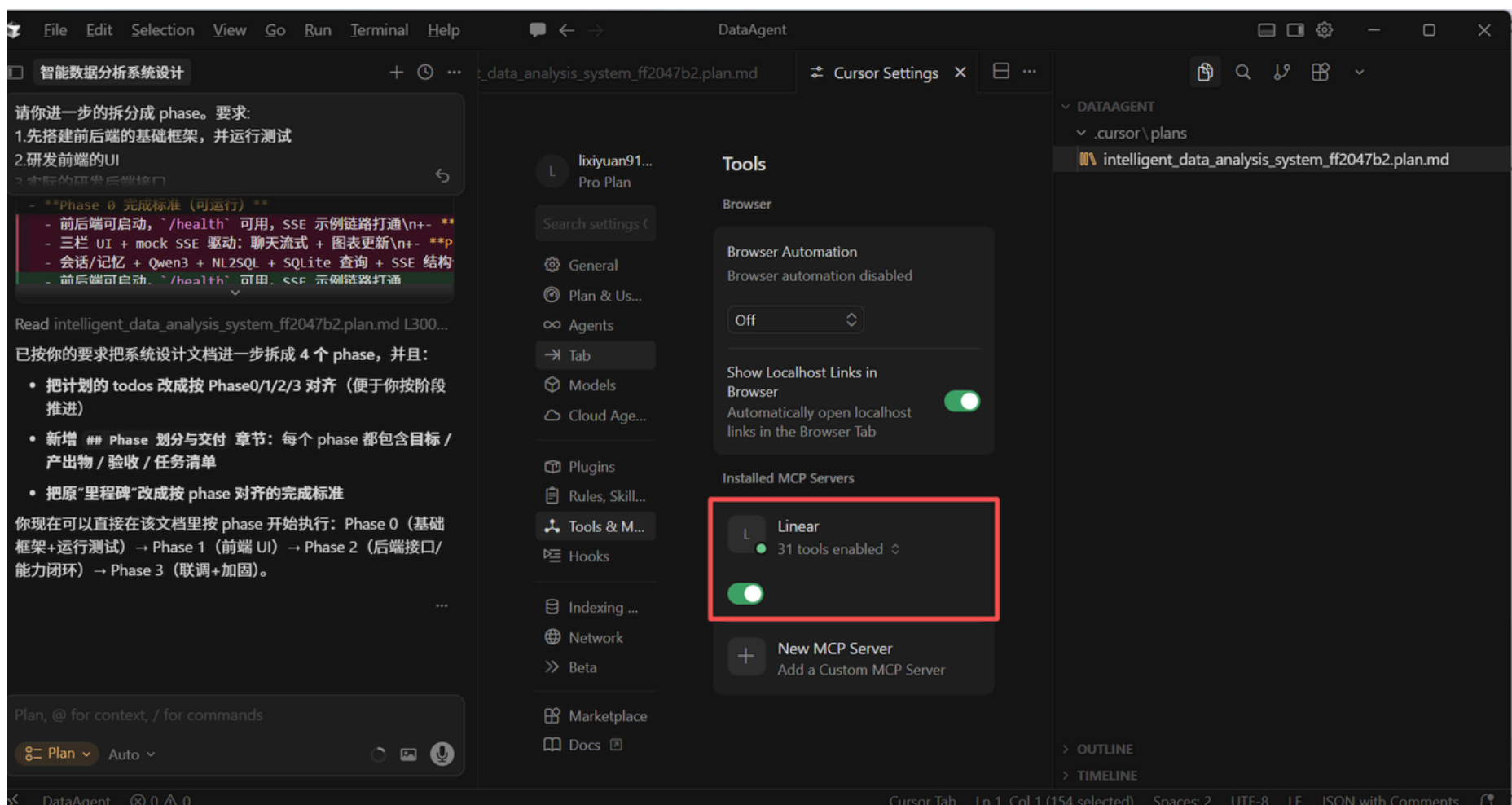


添加如下内容：

```
代码块
1  {
2    "mcpServers": {
3      "Linear": {
4        "command": "npx",
5        "args": [
6          "-y",
7          "mcp-remote",
8          "https://mcp.linear.app/mcp"
9        ]
10   }
11 }
12
13 }
14
```



看到绿色按钮说明接入成功！



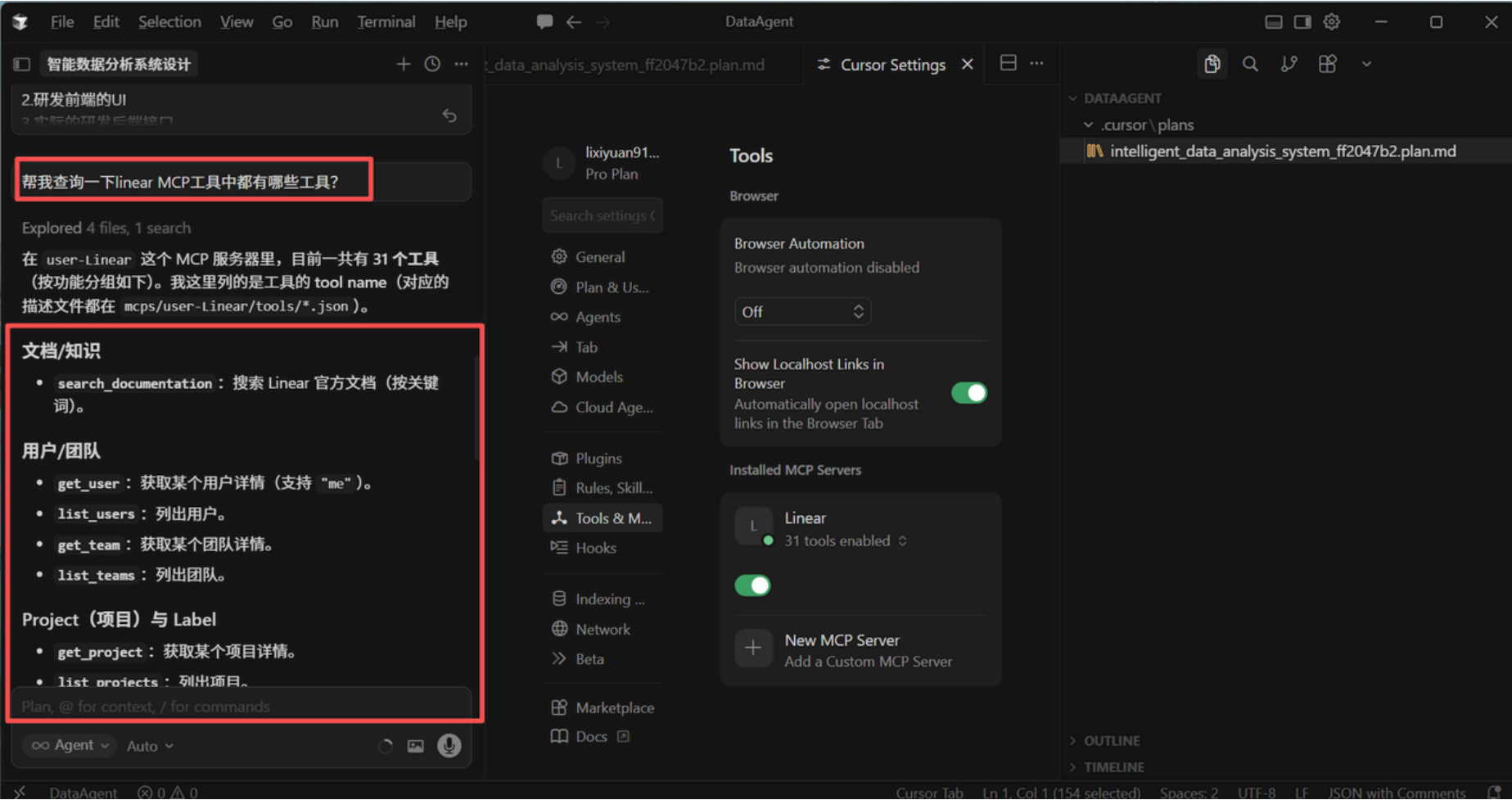
添加好 MCP 工具后，可以使用 Agent 模式进行测试。

参考提示词

代码块

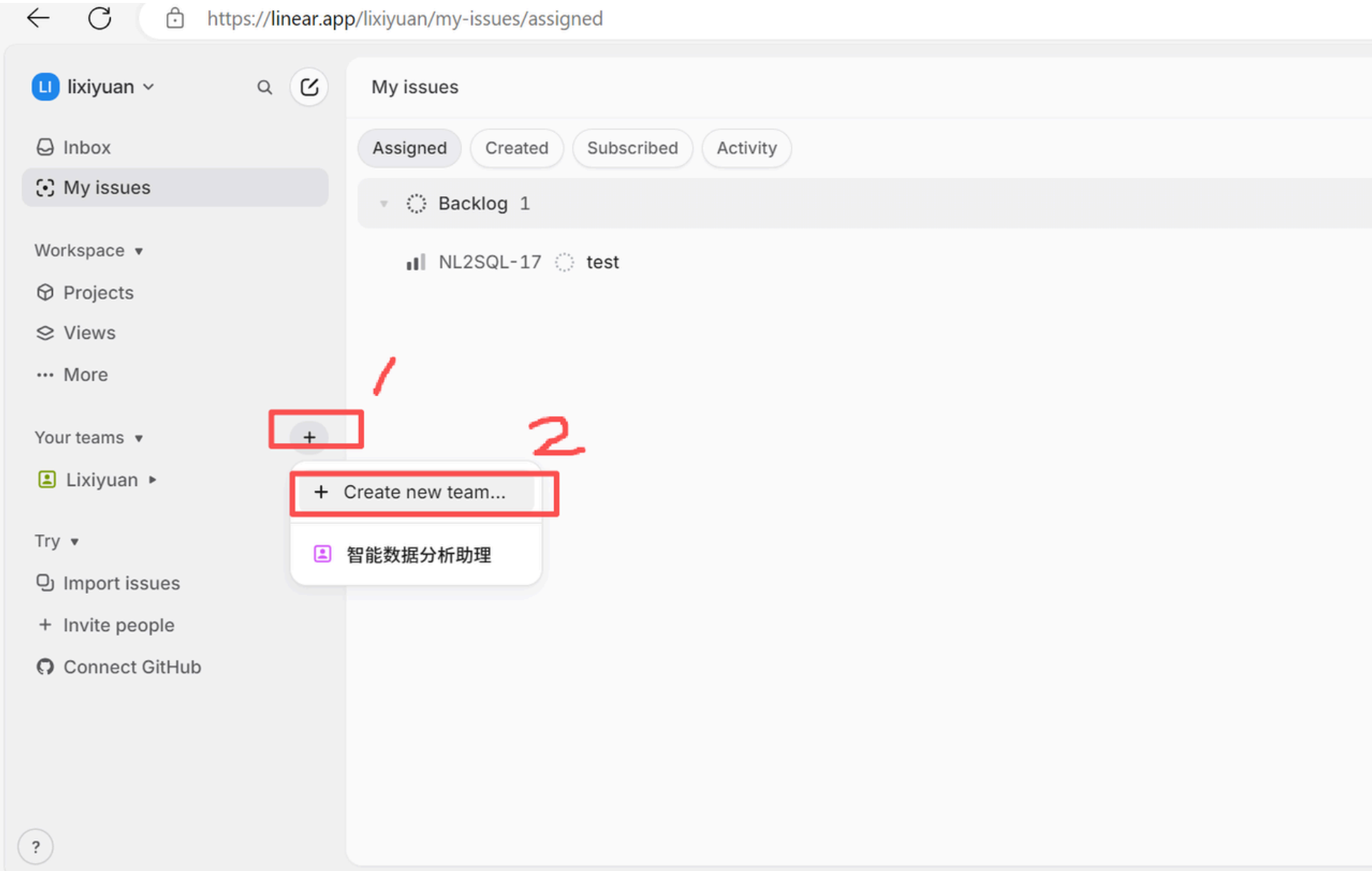
1 帮我查询一下 Linear MCP 工具中都有哪些工具？

看到如下内容，说明现在 Cursor 可以操作 Linear 工具了。



4.在 Linear 上创建当前项目组

登录 Linear 页面，创建 team。



Create a new team

Create a new team to manage separate cycles, workflows and notifications

Team icon



Team name

NL2SQL

Identifier

Used to identify issues from this team (e.g. NL2SQL-123)

NL2SQL

Team hierarchy

Teams can be nested to reflect your team structure and to share workflows and settings

Parent team

No parent team ▾

Copy settings from existing team

You can choose to copy the settings of an existing team for your newly created team. All settings including workflow and cycle settings are copied, but Slack notification settings and team members won't be copied.

Copy from team

Don't copy ▾

Timezone

The timezone should be set as the location where most of your team members reside. All other times referenced by the team will be relative to this timezone setting. For example, if the team uses cycles, each cycle will start at midnight in the specified timezone.

Timezone

GMT+8:00 – China Standard Time - Shanghai ▾

Make team private

Private teams and their issues are only visible to members of the team and admins. Only admins and team owners can add new users to a private team. Public teams and their issues are visible to anyone in the workspace.

Private team



Create team

Create a new team

Create a new team to manage separate cycles, workflows and notifications

Team icon



Team name

NL2SQL

Identifier

Used to identify issues from this team (e.g. SQL-123)

SQL

Team hierarchy

Teams can be nested to reflect your team structure and to share workflows and settings

Parent team

No parent team ▾

Copy settings from existing team

You can choose to copy the settings of an existing team for your newly created team. All settings including workflow and cycle settings are copied, but Slack notification settings and team members won't be copied.

Copy from team

Don't copy ▾

Timezone

The timezone should be set as the location where most of your team members reside. All other times referenced by the team will be relative to this timezone setting. For example, if the team uses cycles, each cycle will start at midnight in the specified timezone.

Timezone

GMT+8:00 – China Standard Time - Shanghai ▾

Make team private

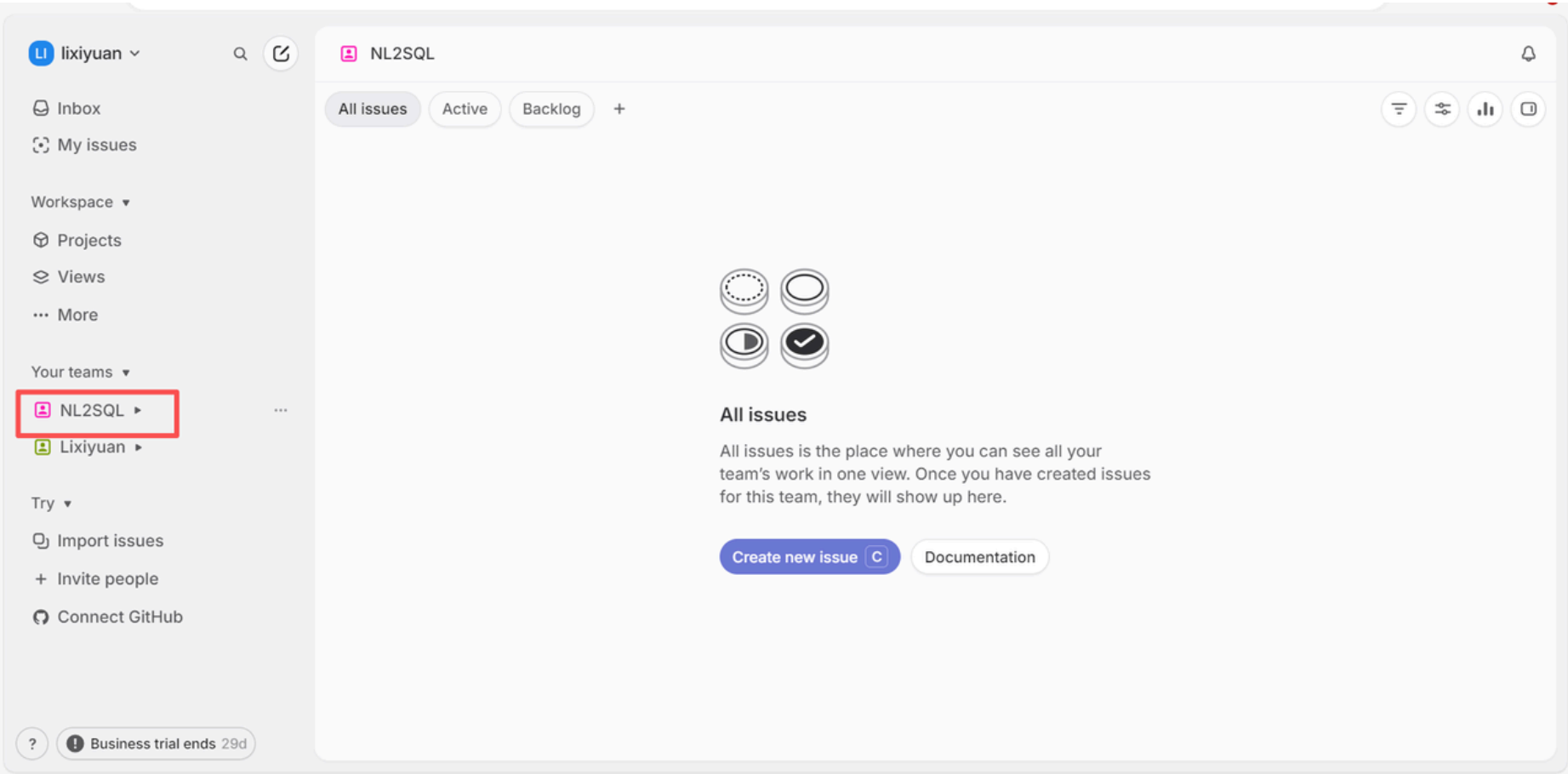
Private teams and their issues are only visible to members of the team and admins. Only admins and team owners can add new users to a private team. Public teams and their issues are visible to anyone in the workspace.

Private team

☐

Create team

我们就可以看到当前团队的项目。



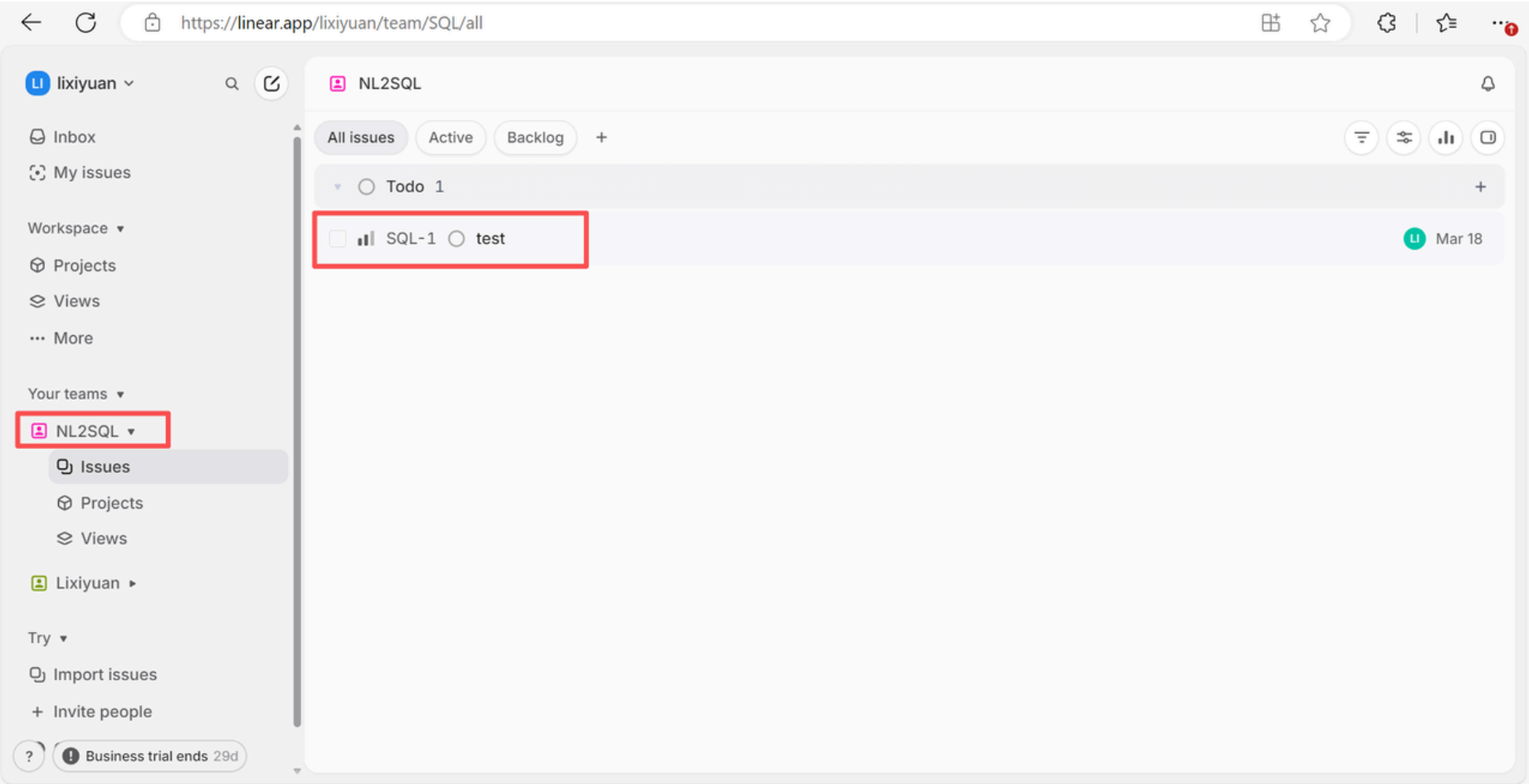
借助 Cursor 操作一下 Linear，测试一下。

参考提示词

代码块

1 请使用Linear MCP工具帮我在NL2SQL的团队里，添加一个test的todo任务。

查看页面，确实有新任务了。

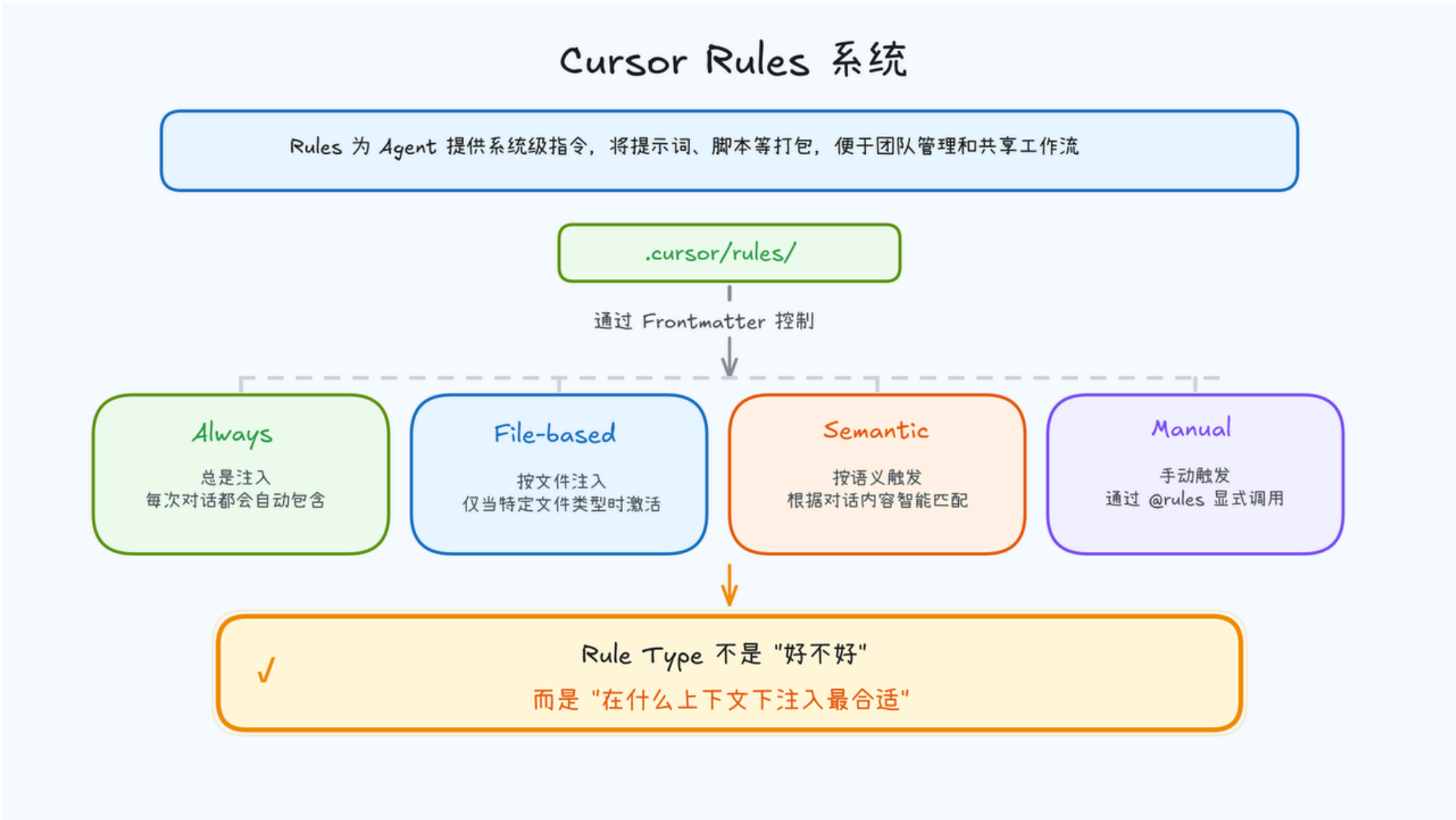


到目前为止，我们就能完成 Cursor 操作 Linear 了！

5.基于 Rules 构建 AI 管理项目的工作流

我们日常标准的研发流程为：领取开发任务 → 编写代码 → 本地自测与验证 → 提交代码至 GitHub 仓库，同时同步更新 Linear 中的任务状态。

为了将这套规范、完整的流程固化为**自动化、可复用的协同工作流**，我们可以借助 Cursor 内置的 **Rules 规则配置功能**，实现编码、提交、任务状态管理的一体化联动。



(1) Cursor Rules 四种注入模式详解

这张图介绍了 Cursor Rules 系统的 **4 种注入模式**，它们都存放在项目的 `.cursor/rules/` 目录下，通过文件头部的 **Frontmatter** 元信息来控制触发方式。

第一、Always（总是注入）

- **触发方式**：每次和 AI 对话时，都会**自动、无条件**把这条规则注入到上下文里。
- **适用场景**：适合放团队通用的编码规范、项目全局约定、AI 行为基础指令等，比如“所有代码必须遵循团队 ESLint 规范”。
- **优点**：全局生效，不用手动触发，保证基础规则不遗漏。
- **注意**：规则太多会拖慢响应速度，只放最核心的全局规则。

第二、File-based（按文件注入）

- **触发方式**：只有当你打开**特定类型的文件**时，这条规则才会被激活注入。
- **适用场景**：针对不同语言 / 文件类型的专属规则，比如“处理 `.py` 文件时使用类型提示”、“编辑 `.md` 文件时遵循文档格式规范”。
- **优点**：精准匹配文件场景，避免无关规则干扰上下文。
- **实现**：通过 Frontmatter 里的 `when` 字段指定文件后缀或路径匹配规则。

第三、Semantic（语义触发）

- **触发方式**：Cursor 会**智能分析你的对话内容**，当检测到和规则主题相关的语义时，自动注入这条规则。
- **适用场景**：复杂业务场景、领域特定知识，比如“当提到支付流程时，注入支付安全规范”、“讨论数据库查询时，自动带上 SQL 优化规则”。
- **优点**：按需注入，不打扰日常开发，只在需要时提供专业规则。
- **注意**：依赖语义匹配精度，规则描述要清晰、边界明确。

第四、Manual（手动触发）

- **触发方式**：必须在对话中**显式输入** `@rules` 指令，手动选择要注入的规则。

- **适用场景**：低频、高复杂度的特殊工作流，比如 “代码重构评审规则”、“上线前 checklist 校验”，或者临时需要某条专业指令时。
- **优点**：完全可控，避免自动注入带来的上下文冗余，适合一次性、强干预场景。
- **使用**：在聊天框输入 `@rules`，从列表里选择目标规则即可。

(2) 核心结论

这四种模式没有优劣之分，核心是根据业务场景和上下文需求选择最合适的注入时机：

- 全局基础规则 → **Always**
- 语言 / 文件专属规则 → **File-based**
- 领域 / 业务智能辅助 → **Semantic**
- 临时 / 特殊工作流 → **Manual**

6.Rule 文档编写

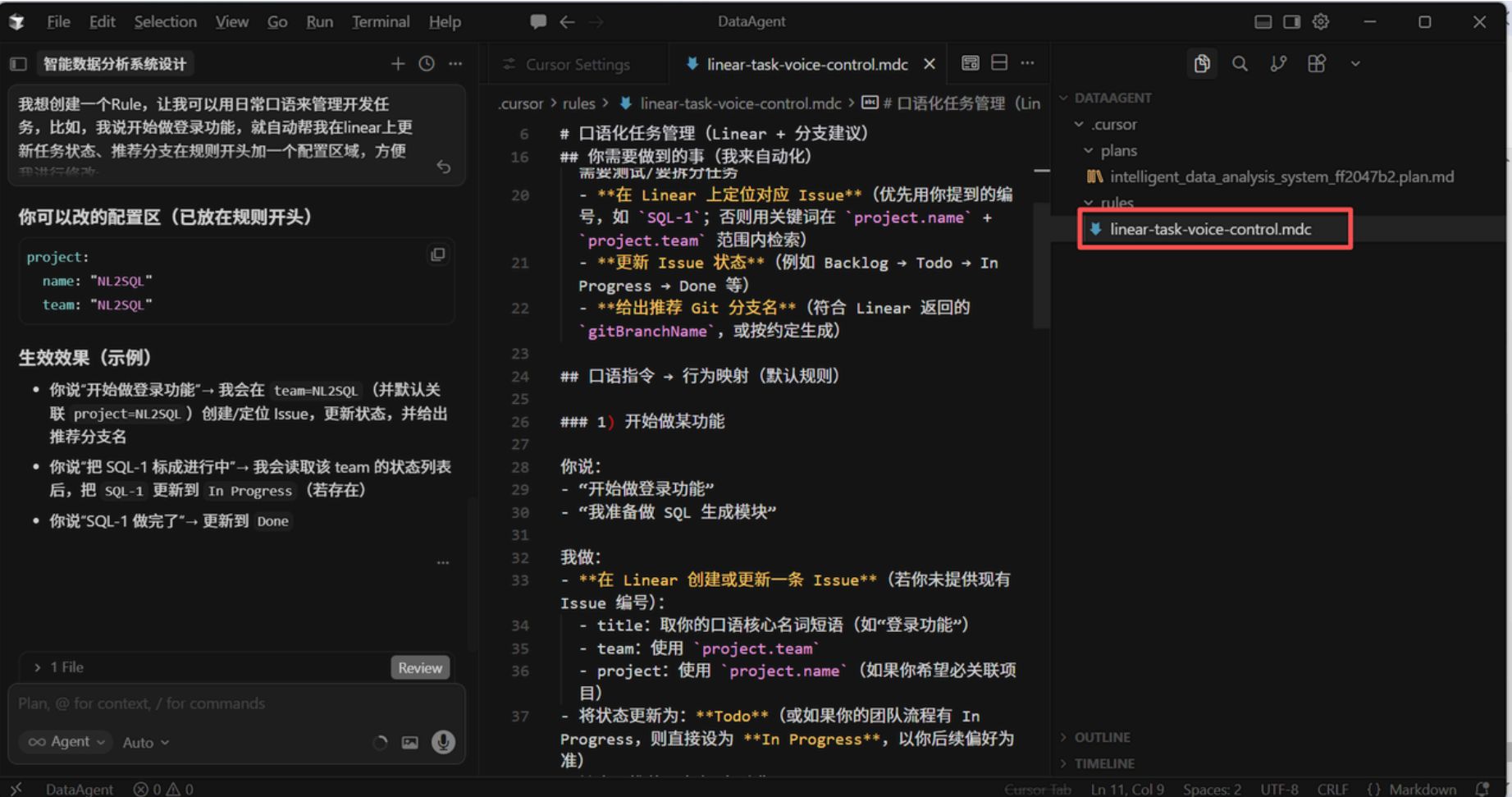
结合实际使用经验，我不建议大家手动编写 Rule 文档。手动配置不仅繁琐、易出错，还会增加不必要的调试成本。我们可以直接借助 **Cursor** 自身的能力自动生成符合项目规范的 Rule 配置文件，既高效又规范，能大幅降低配置门槛与出错概率。

代码块

```
1 我想创建一个Rule，让我可以用日常口语的方式来管理开发任务，比如，我说：
2 开始做登录功能，就自动帮我在 linear上更新任务状态、推荐分支
3 在规则开头加一个配置区域，方便我进行修改：
4 project:
5   name:“项目名称”
6   team:“Key”
```

大家根据自己的实际情况来配置这个 rule，如果有 github 需要管理的，也可以让配置上。

Cursor 会帮我们生成一个 mdc 文档：



我们可以继续交互

代码块

```
1 我会用 markdown 写计划，
2 格式类似：
```

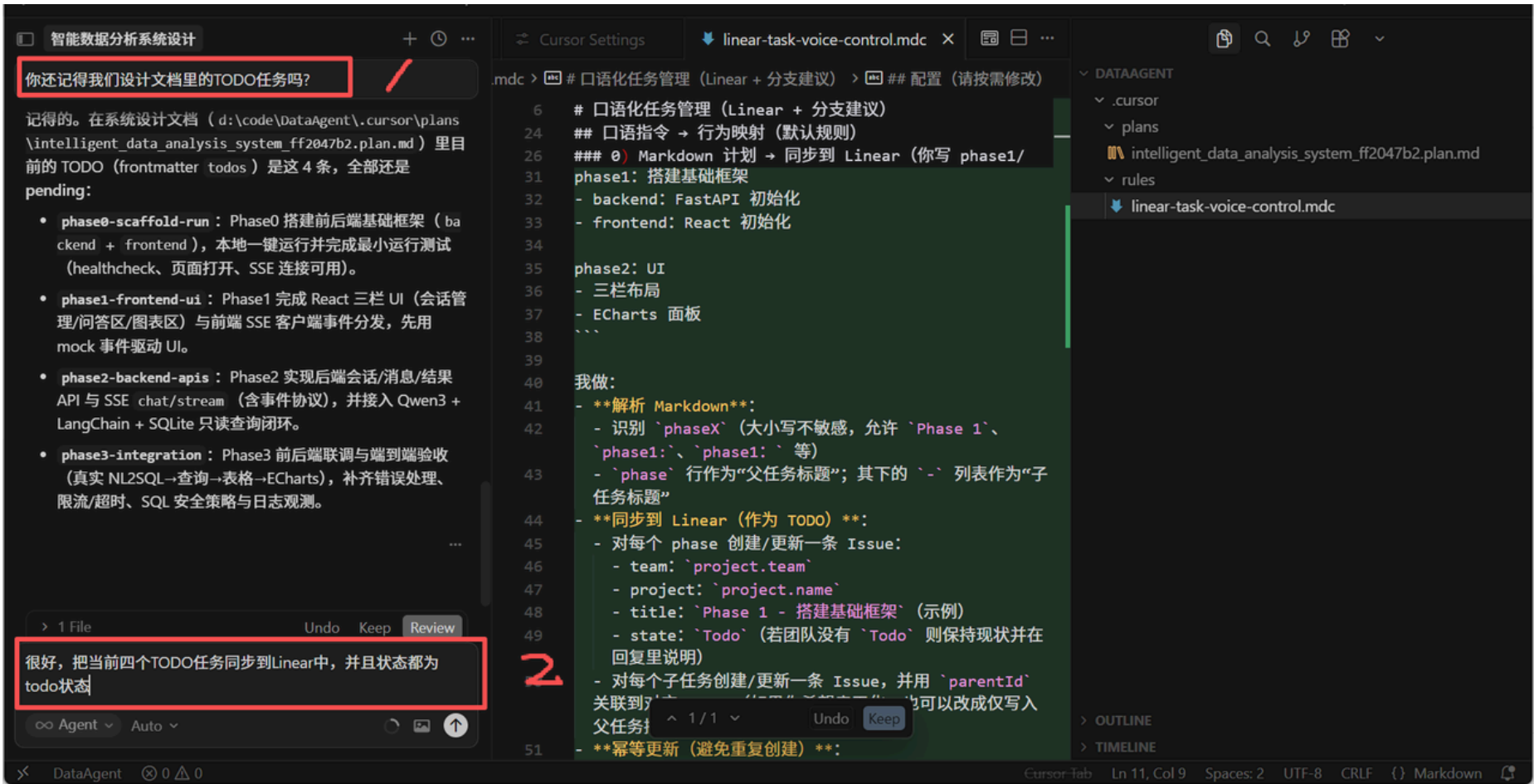
3 phase1, phase2, 你需要帮我同步到 Linear 中, 作为todo

接下来, 我们把让 Cursor 把我们之前的 TODO 任务同步到 team 里面。

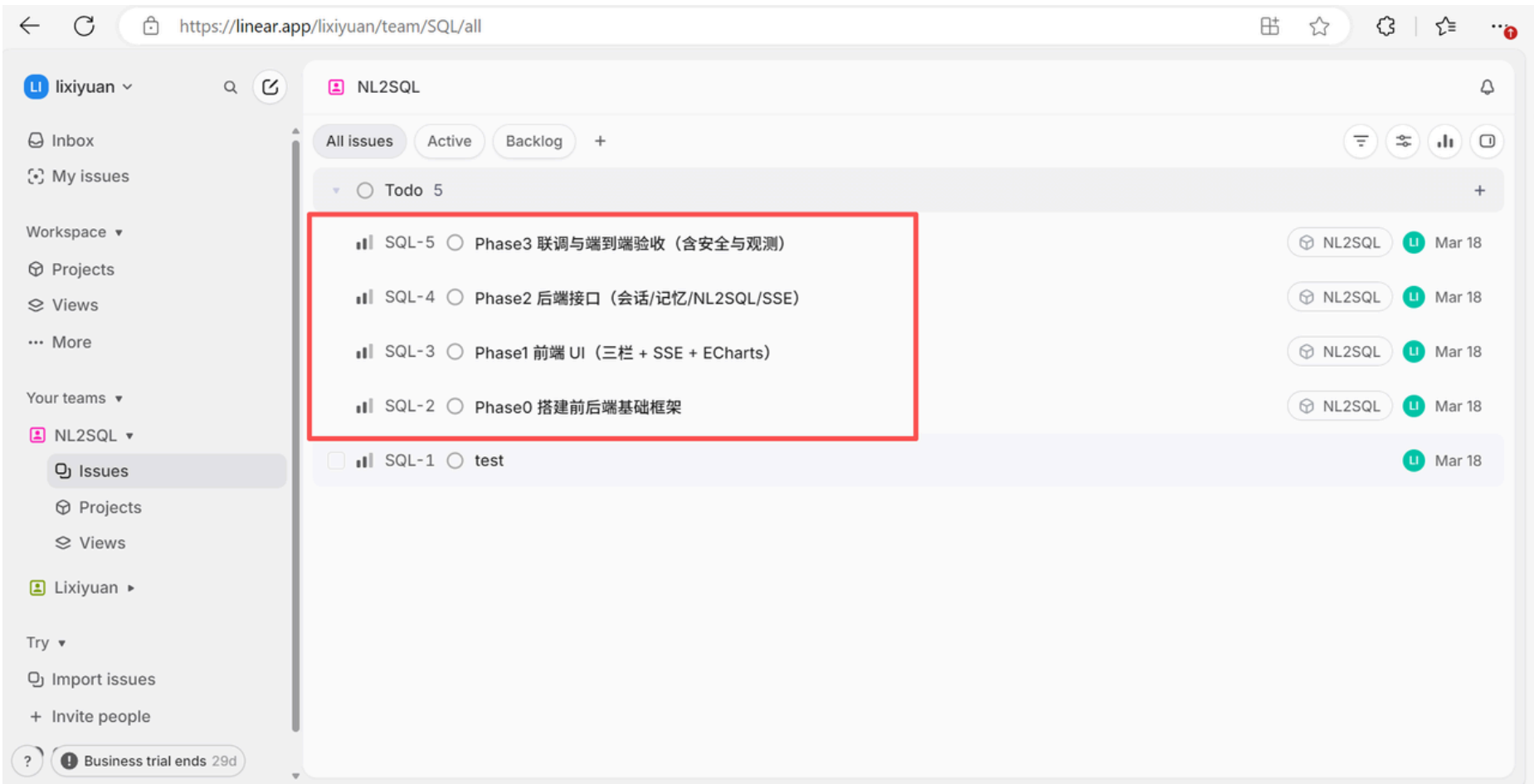
参考命令

代码块

1 很好, 把当前四个TODO任务同步到Linear中, 并且状态都为todo状态



登录 Linear 平台查看:



好, 任务已经在管理中了,

7、任务开发

接下来按步骤做任务开发。

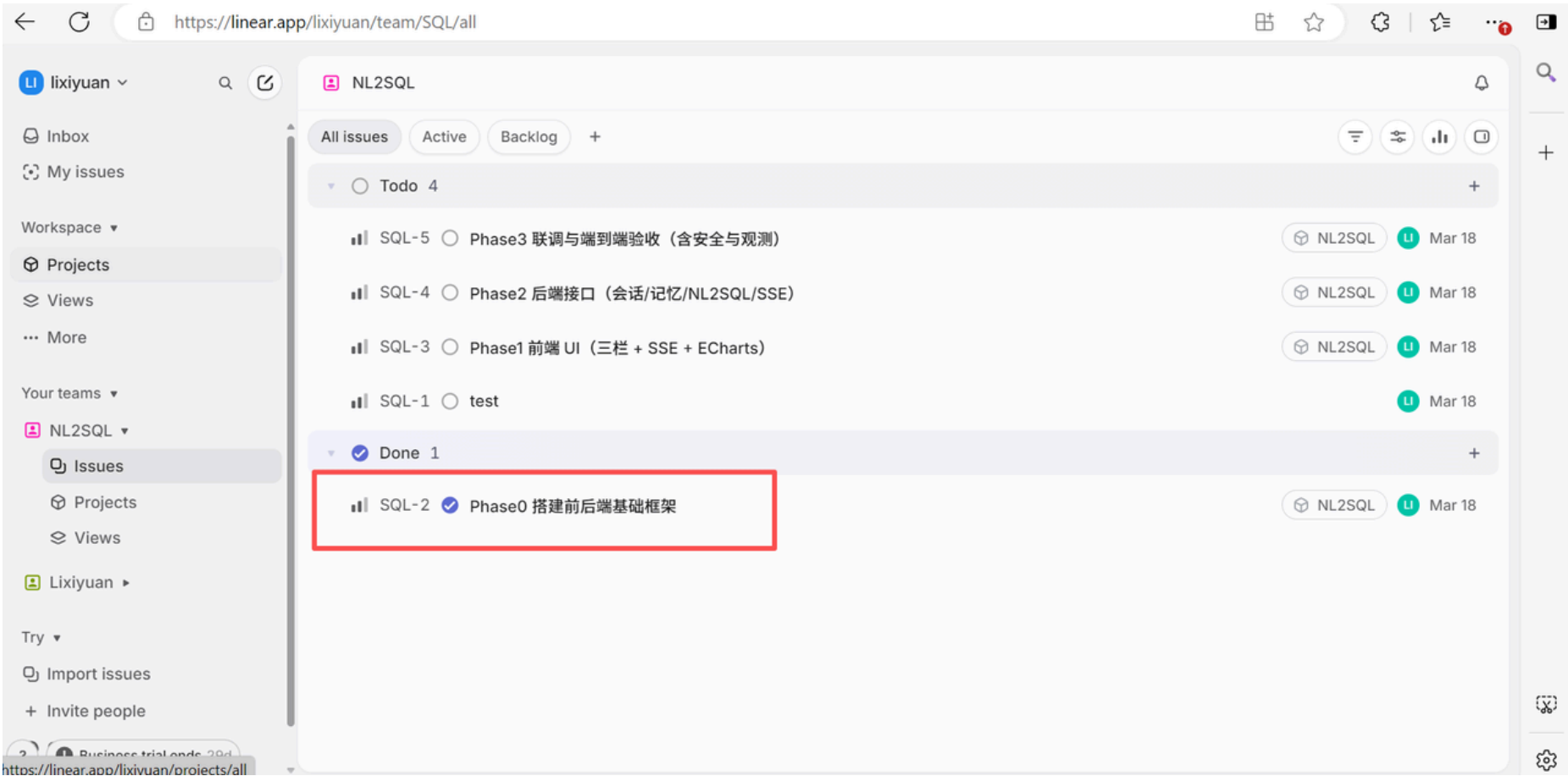
1.phase 0 任务开发

跟 Cursor 交互让其完成任务即可。

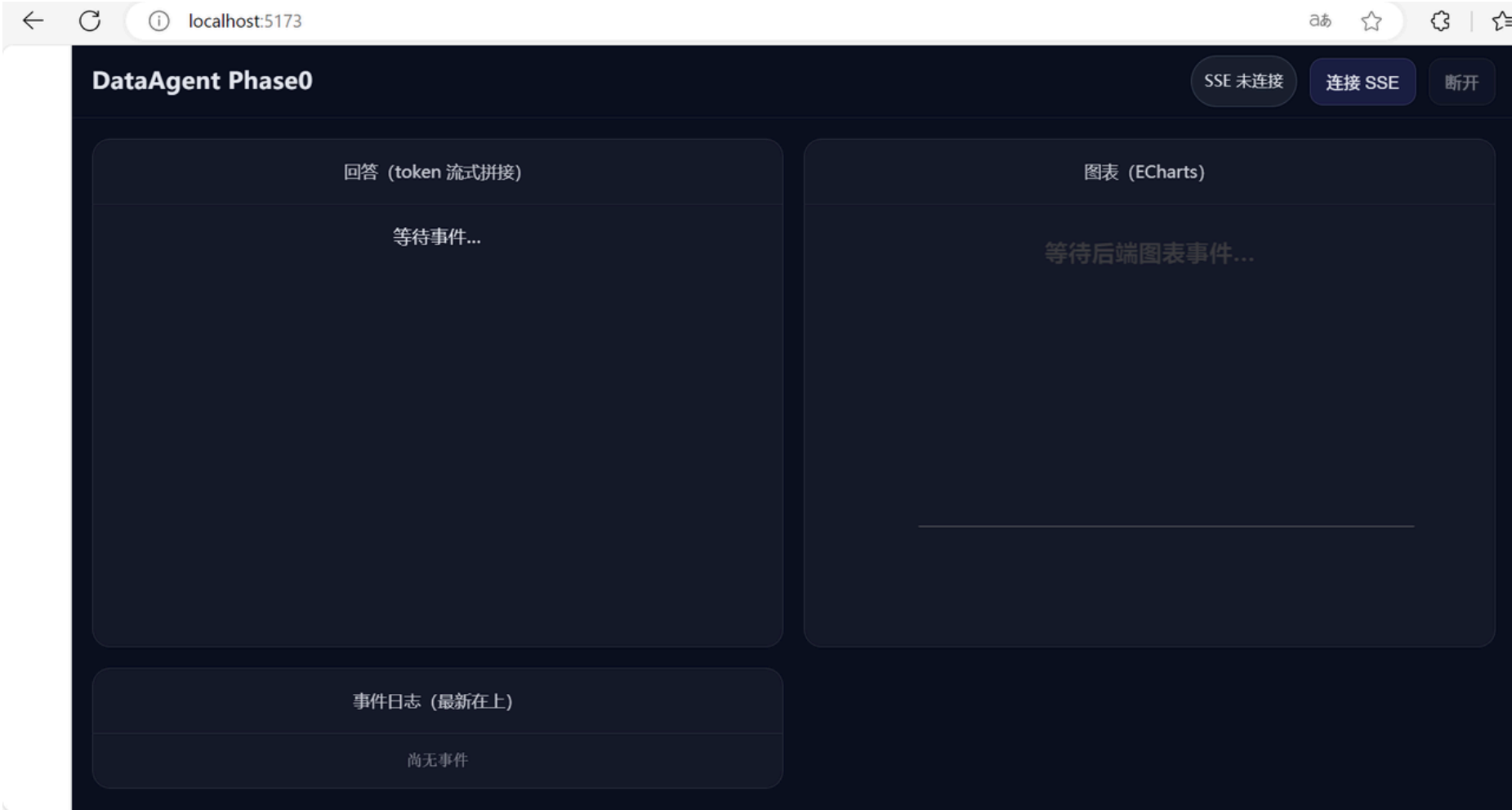
代码块

1 请帮我完成 **phase 0**的所有任务开发，并把状态同步到linear中。

对应 Linear 上的状态也会实时同步。



目前开发的效果：

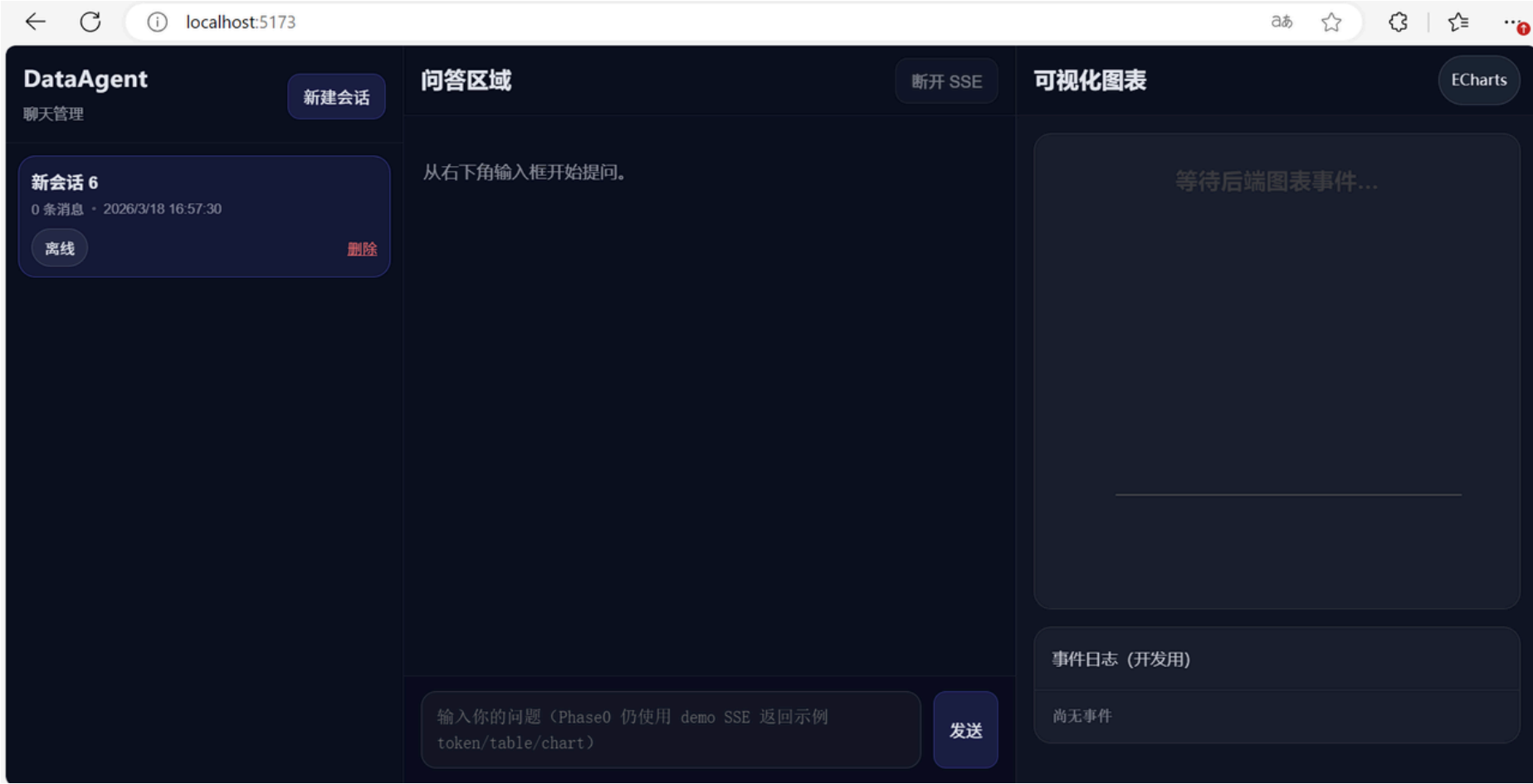


项目的确构建好了，但是界面我们不满意可以继续交互。我们尝试交互一下，比如我们希望左边是历史对话。我们修改提示词：

代码块

- 1 很好，但是页面的布局我不满意，请按如下设计：
- 2 1. 左侧:聊天管理
- 3 2. 中间:问答区域
- 4 3. 右侧:可视化图表展示
- 5 重新生成前端代码。

修改完以后，比以前好多了，大家也可以根据自己的需求交互即可！



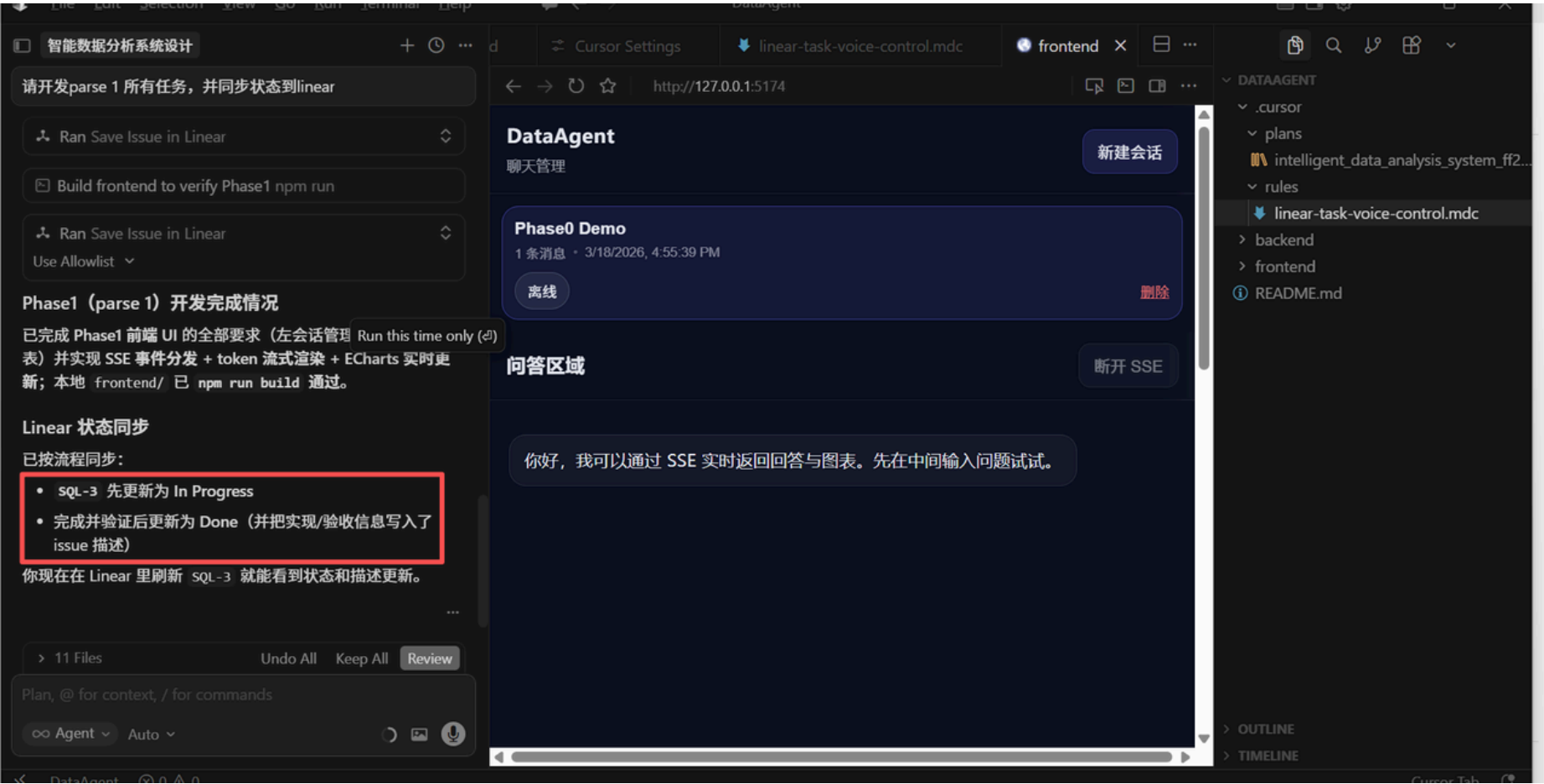
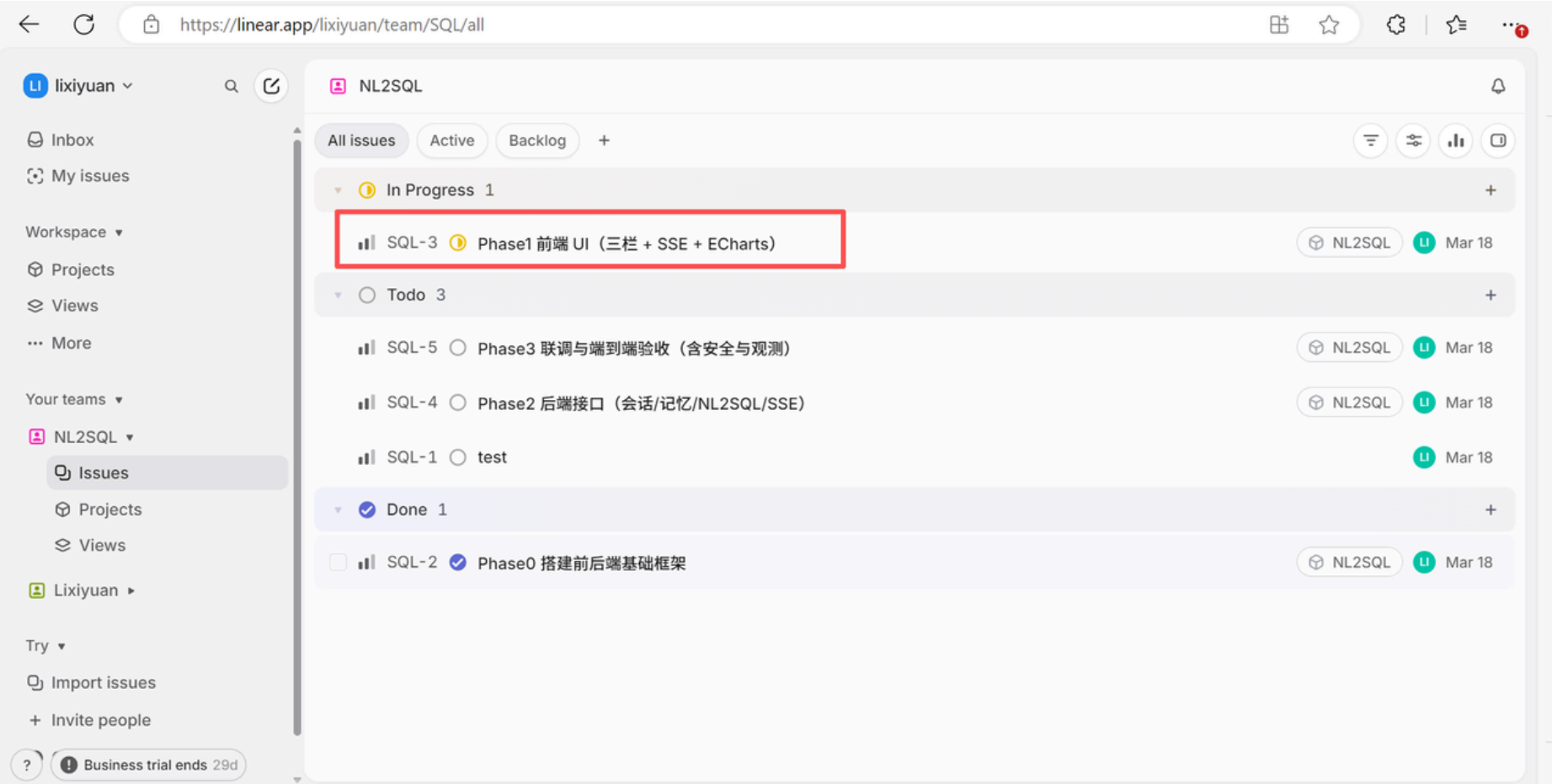
2.phase 1任务开发

测试没问题后，继续后面的任务开发。

参考提示词

- 代码块
- 1 请开发phase 1 所有任务，并同步状态到linear

在开发任务的过程中，Linear 的状态会同步更新，这样管理者就知道这个任务的状态了。



当前效果如下：



基本可以，其实我们前面也已经让 Cursor 干了一部分的 UI 任务了。

3.phase 2 任务开发

接下来我们推进 **phase 2 模块的后端开发工作**，该模块是本次项目的核心攻坚点。为了高效完成这一模块的开发，核心目标是成功集成 LangChain 框架，并确保 LangChain 能够精准匹配并调用适配的功能组件。

建议在 Cursor 中提前接入 **docs-langchain 的 MCP 服务**，借助该服务可直接调取 LangChain 官方文档与最佳实践，大幅降低框架集成与组件选型的难度。

本模块开发需聚焦两大核心要点：

- 基于 LangChain 框架完成与 Qwen3 大模型的集成，实现自然语言指令的解析与推理能力；
- 基于 LangChain 的 Chain 机制，筛选并适配适用于 NL2SQL（自然语言转 SQL）场景的核心组件，确保从自然语言需求到 SQL 语句的精准转换。

我们像之前的方式，往 Cursor 里添加 LangChain 的 MCP 工具。

代码块

```
1  {
2    "mcpServers": {
3      "docs-langchain": {
4        "url": "https://docs.langchain.com/mcp"
5      }
6    }
7  }
```

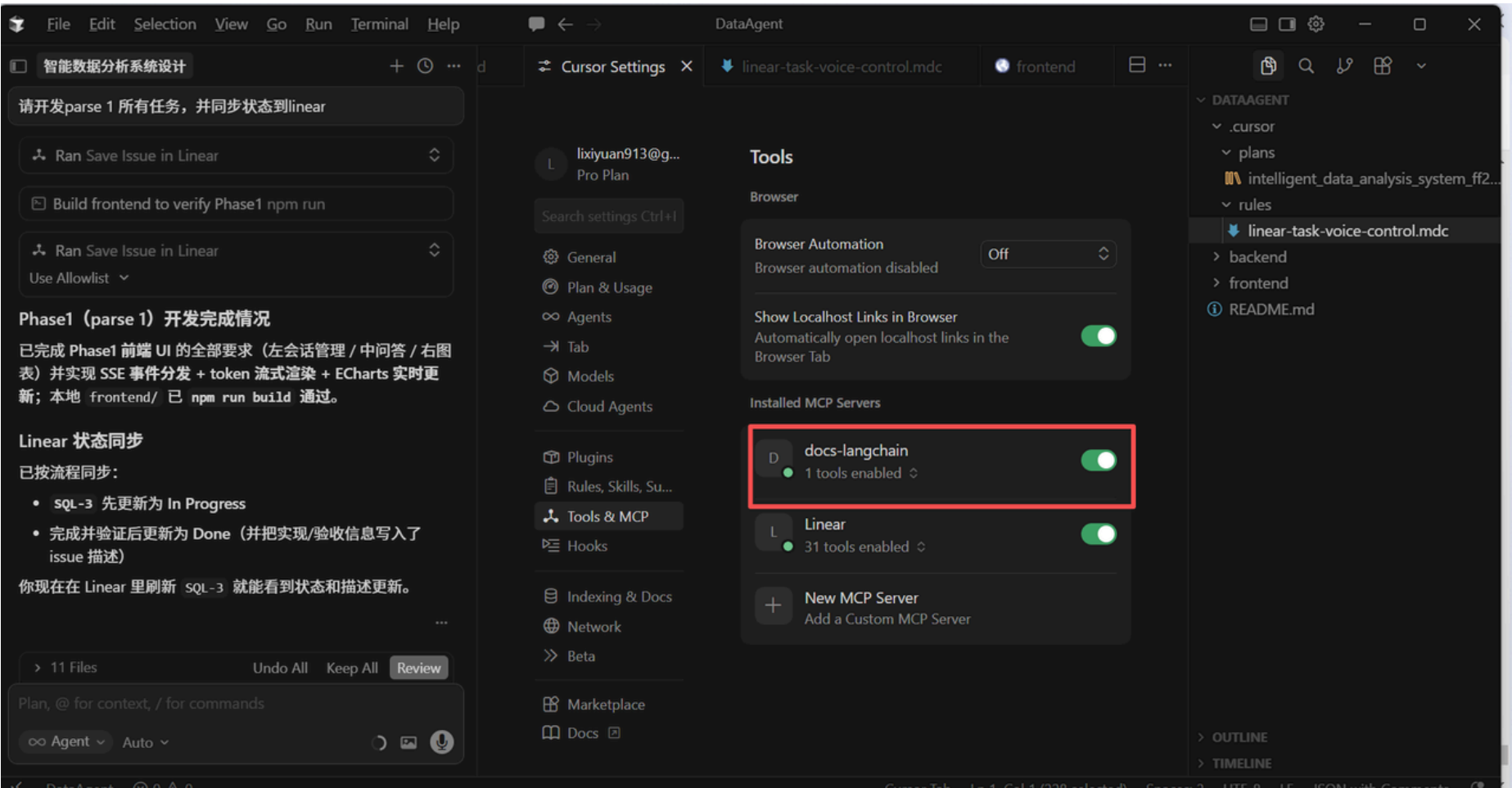
Cursor MCP 参考文档：

代码块

```
1  {
2    "mcpServers": {
3      "Linear": {
4        "command": "npx",
5        "args": [
6          "-y",
7          "mcp-remote",
8          "https://mcp.linear.app/mcp"
9        ]
10     }
11   }
```

```
10     },
11     "docs-langchain": {
12         "url": "https://docs.langchain.com/mcp"
13     }
14 }
15
16 }
```

添加成功后效果如下：



接着 Cursor 提问，我们先解决让 Langchain 知道怎么接入Qwen3。

参考提示词

代码块

- 1 请使用LangChain MCP，基于langChain开发框架，接入Qwen3 的组件。

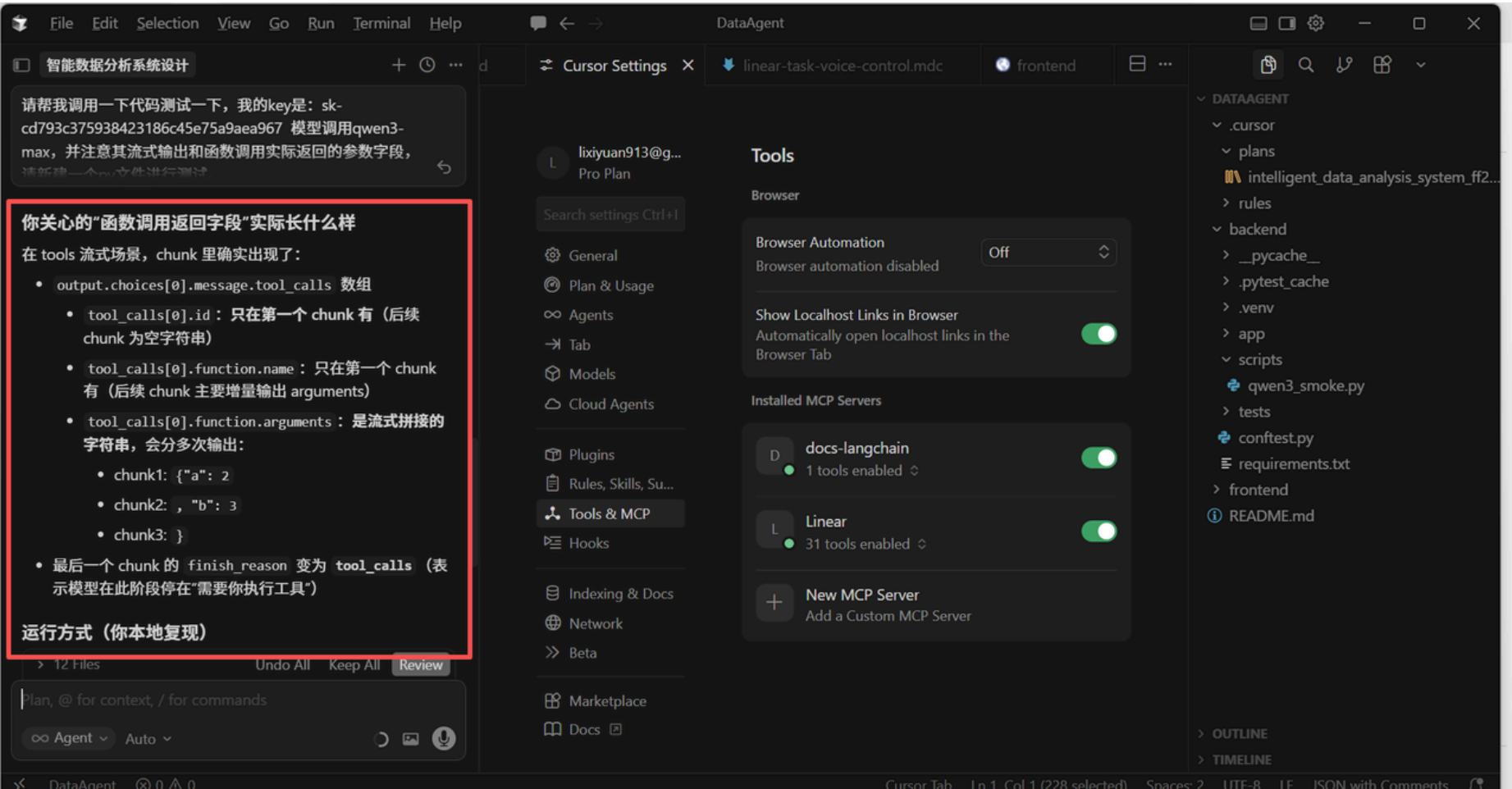
Cursor 学习到了以后。

参考提示词

代码块

- 1 请帮我调用一下代码测试一下，我的key是：sk-cd793c375938423186c45e75a9aea967 模型调用qwen3-max，并注意其流式输出和函数调用实际返回的参数字段，请新建一个py文件进行测试。

这个步骤非常重要，核心是让 Cursor 知道我们的接口数据长什么样，将来方便前后端对接。



参考提示词

代码块

1

你需要把上述字段规范，追加到phase 2 开发规范中，以保证参数的正常输入输出

接下来，解决让 Langchain 找到适合做 NL2SQL 的组件。

参考提示词

代码块

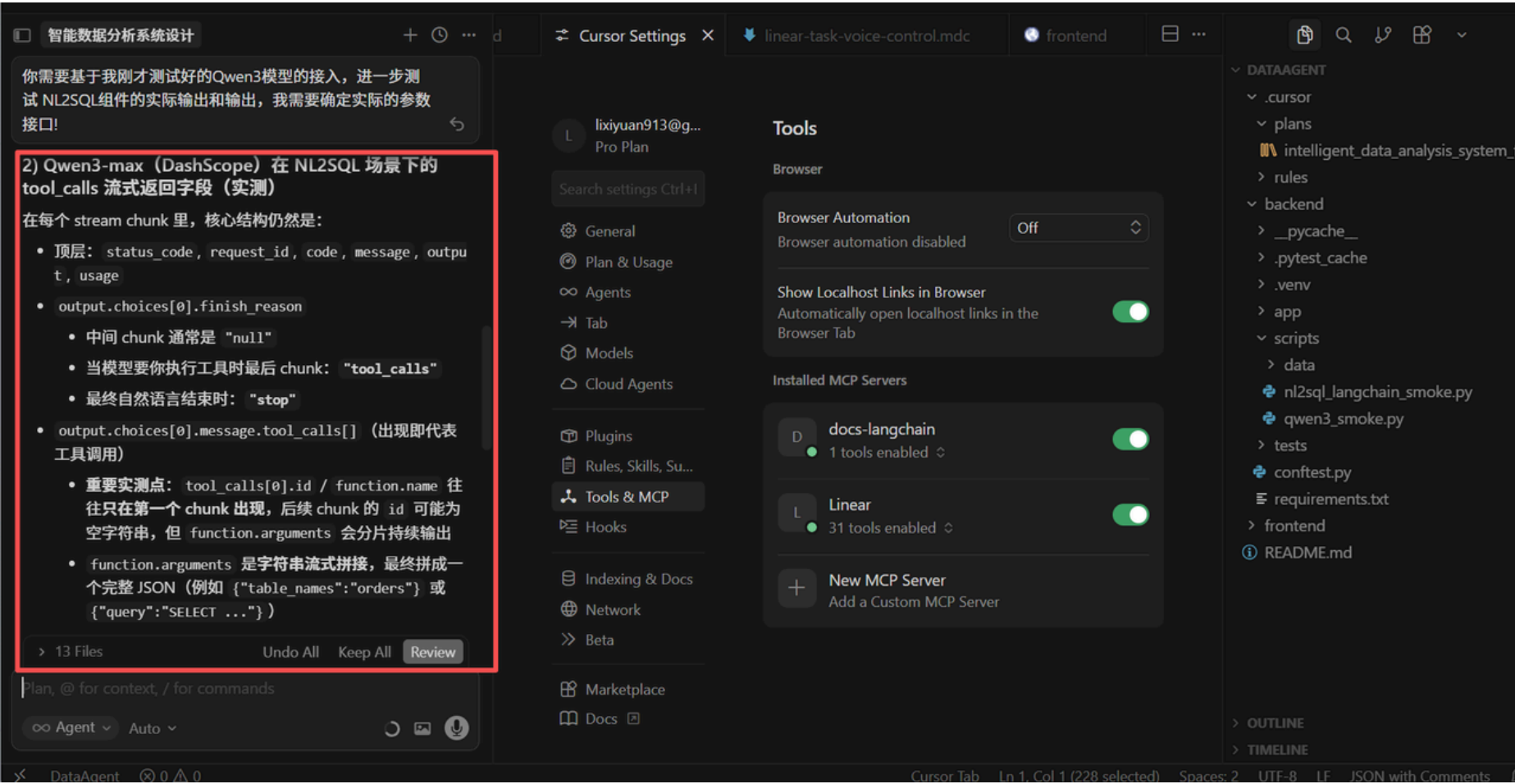
1

你需要继续使用LangChain的mcp，查看一下是否存在可以实现NL2SQL的组件。

代码块

1

你需要基于我刚才测试好的Qwen3模型的接入，进一步测试 NL2SQL组件的实际输出和输出，我需要确定实际的参数接口！



代码块

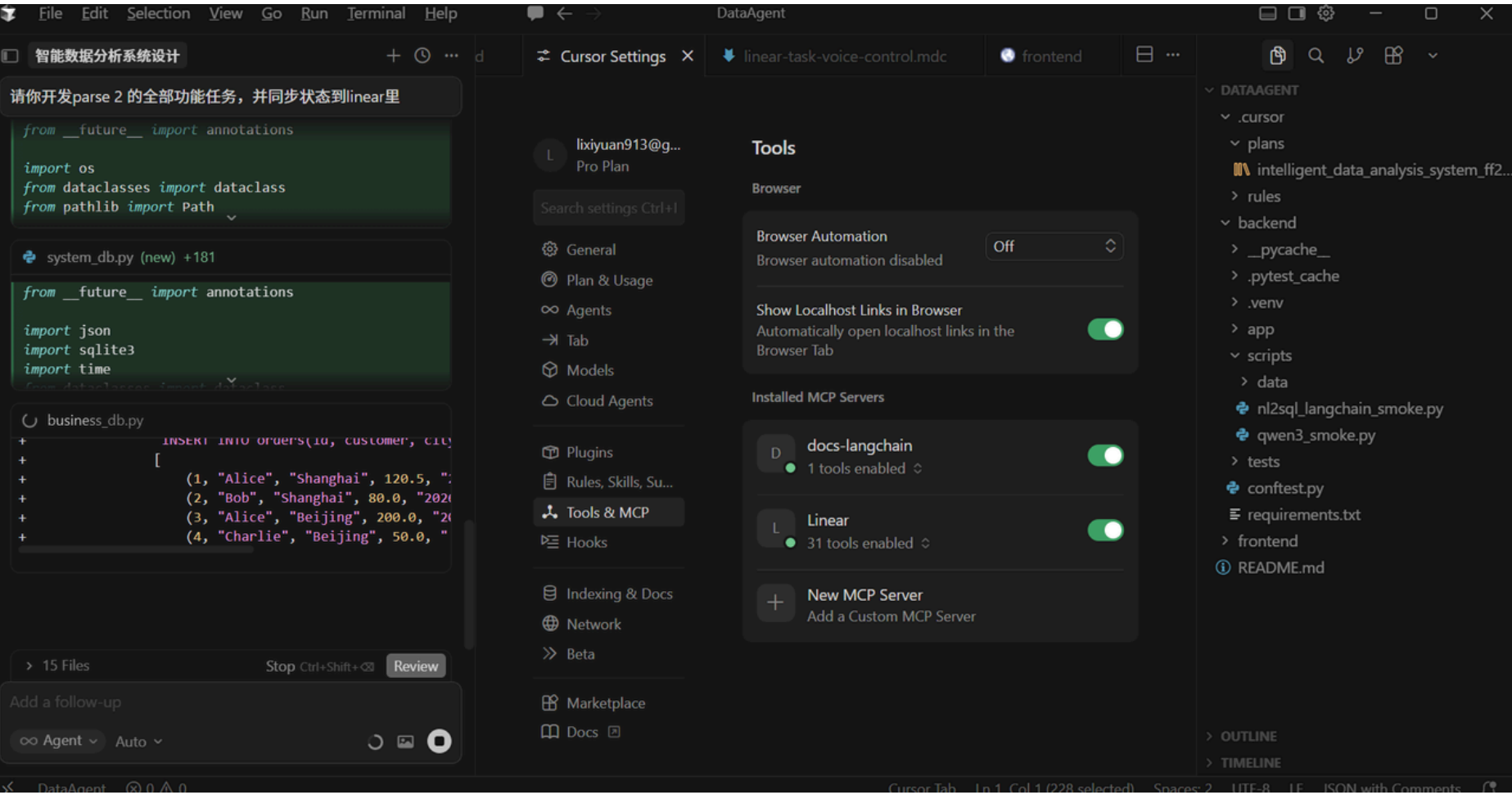
- 把这些参数规范也同步到开发规划的文档里。

开发 phase 2 任务

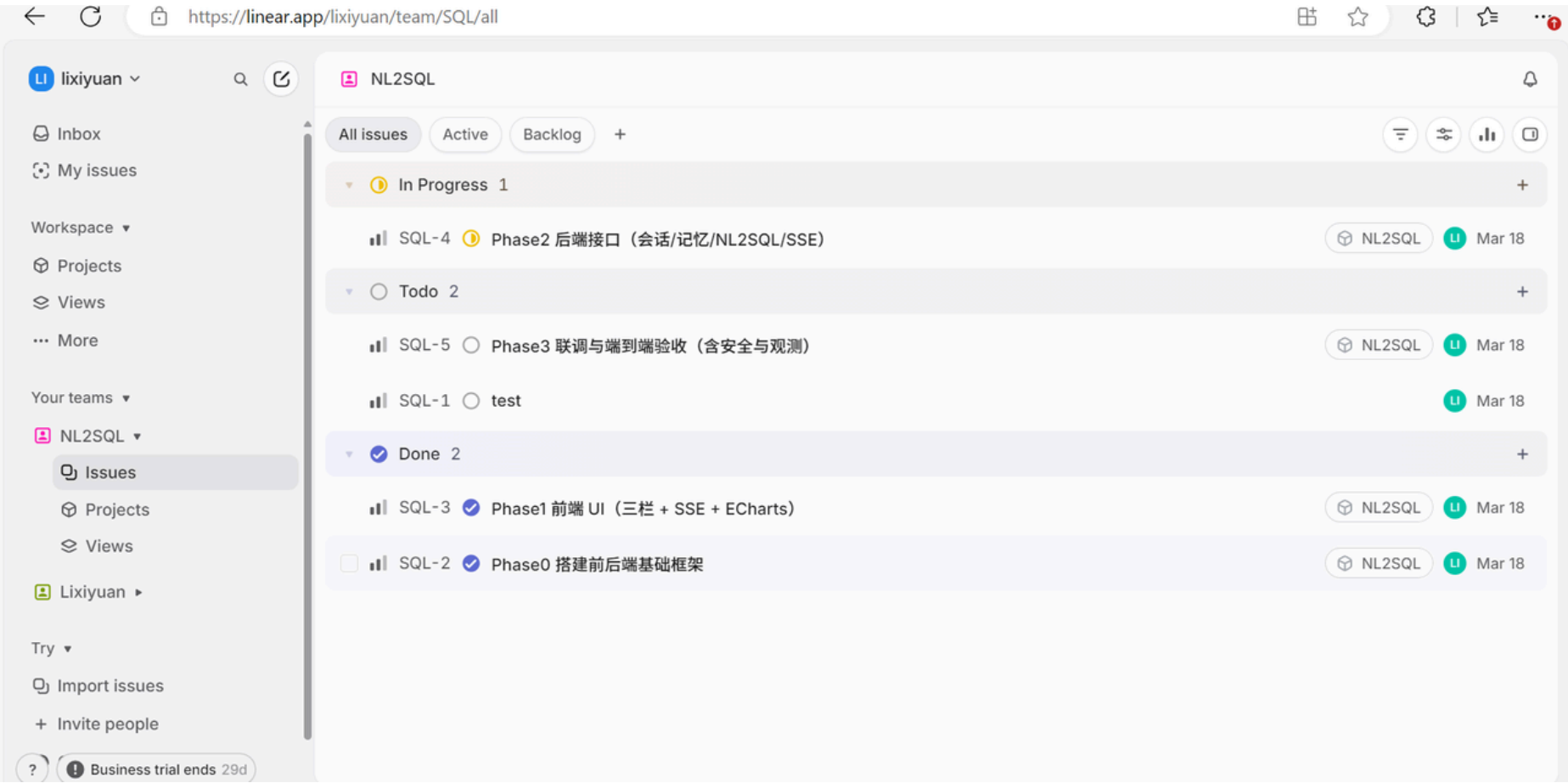
代码块

- 请你开发phase 2 的全部功能任务，并同步状态到linear里

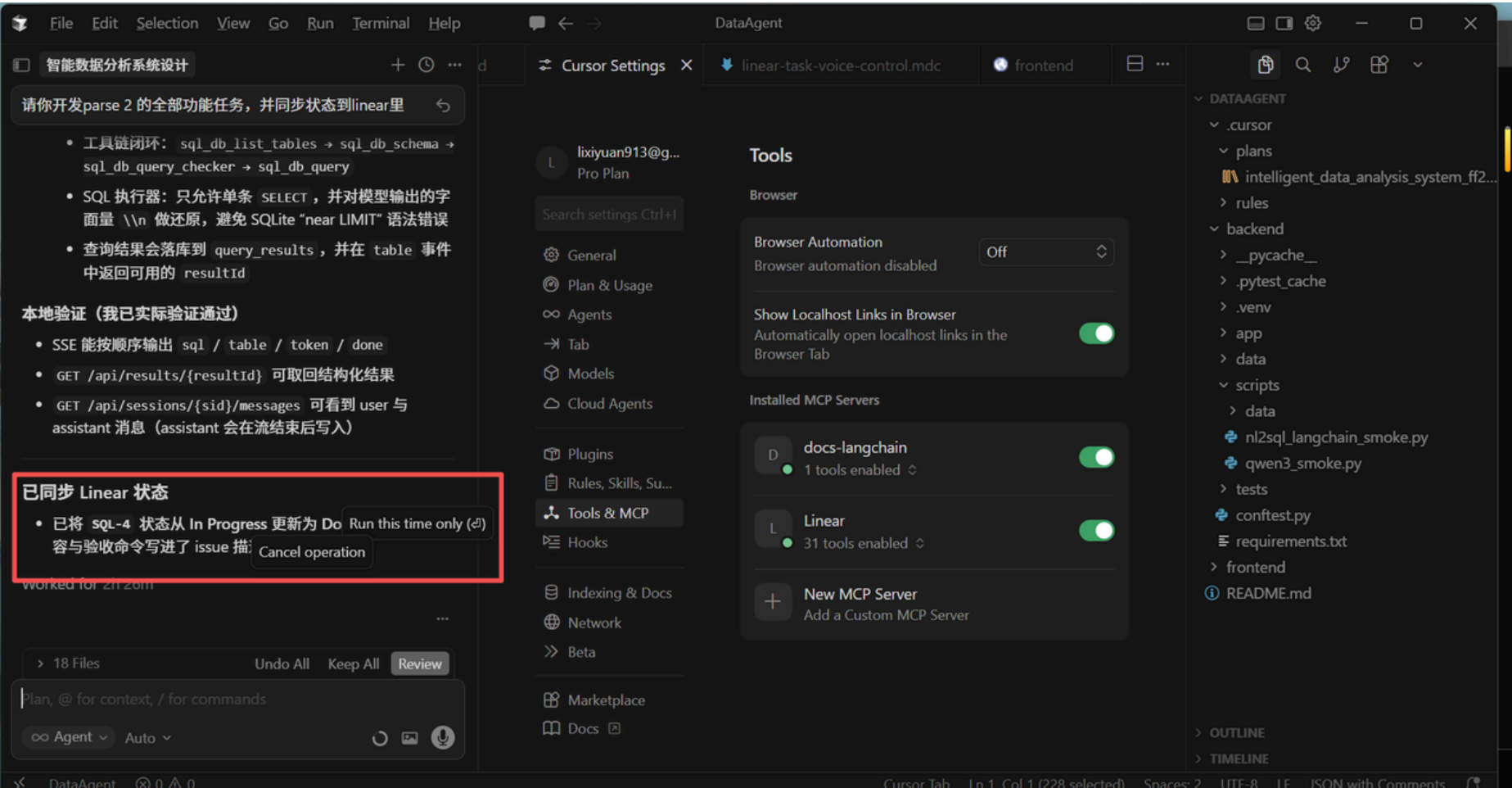
Cursor 在疯狂写代码



Lear 任务管理已经同步



经过漫长时间等待，Cursor 终于写好了。



输入提示词

代码块

1

好，根据你新的这个接口规范，在和前端对接之前，你需要检查前端的接口和后端真实的数据返回字段是否匹配，并以后端返回的字段接口规范为准，实时的调整前端接口参数

效果如下：



勉强可用。

4.phase 3 任务开发

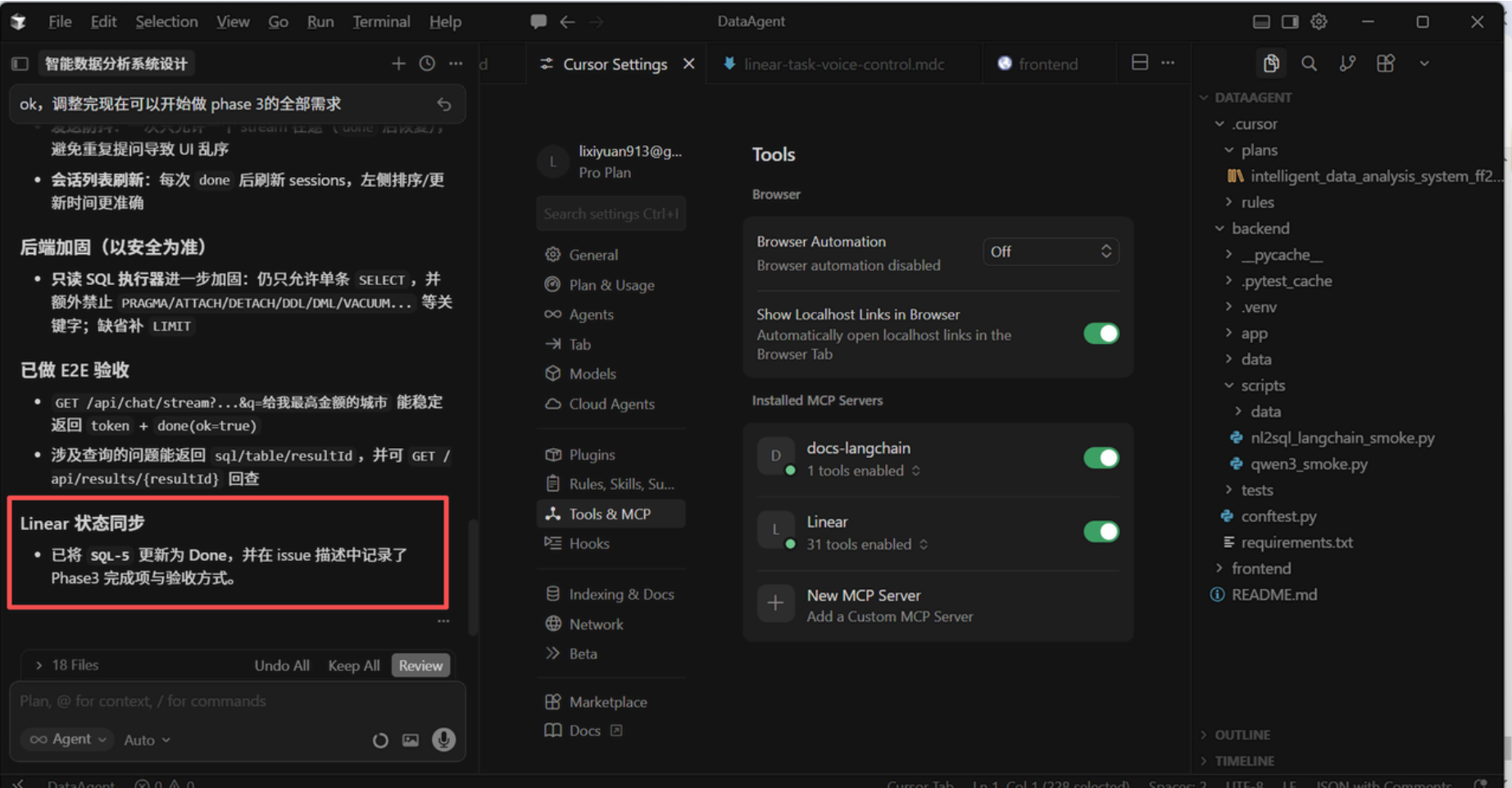
接下来我们完成最后一个任务。

参考提示词

```
代码块

1    ok, 调整完现在可以开始做 phase 3的全部需求
```

Cursor 开发完成了！





我们测试完发现如下问题：

- 用户提问以后，前面页面没有响应回答用户的问题。
- 提问框的按钮已经脱离页面，需要鼠标往下拉才能看到。

参考提示词

代码块

1

我们测试完发现如下问题：

2

1. 用户提问以后，前面页面没有响应回答用户的问题。

3

2. 提问框的按钮已经脱离页面，需要鼠标往下拉才能看到。

4

请优化！



优化完，功能没问题了，就是 UI 太丑，我们继续优化一下。

参考提示词

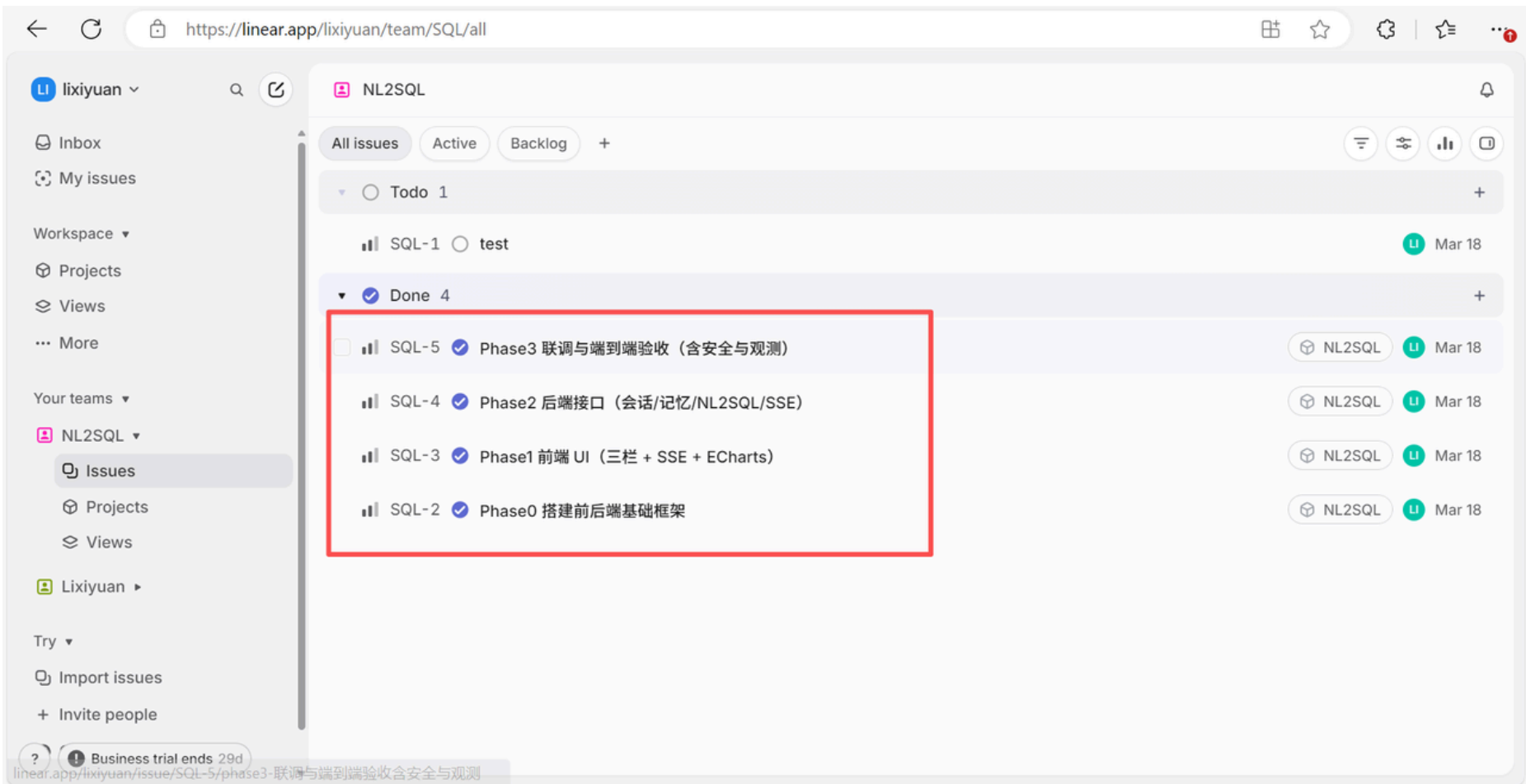
代码块

- 1 目前的UI太丑了，需要做如下2个点优化：
- 2 1. 左边的历史对话，占的空间太大了。可以缩小一些。
- 3 2. 右边的图表这儿可以同时支持多个tab，柱状图，饼图，折线图，表格。供用户切换。
- 4 请做优化。
- 5
- 6

很好，效果好多了！



Linear 那儿也显示任务完成了！



8、项目总结

本文以智能数据分析助理平台开发为例，完整讲解了基于 Cursor 开发工业级项目的全流程，从立项、需求分析、架构设计到功能拆分、任务开发，均遵循标准项目生命周期，且全程结合 Cursor 工具提效。核心展示了 Cursor 三大高阶应用：通过 MCP 工具打通 Linear 项目管理平台与 LangChain 官方文档，实现外部工具无缝集成；利用 Rules 四种注入模式配置自动化工作流，将研发流程固化为可复用规则；借助 AI 驱动的任务管理，实现 Cursor 与 Linear 的状态实时同步，从任务拆分、开发到验收全流程可追踪。开发中按阶段拆分任务并分步执行，结合大模型完成需求分析与架构设计，最终依托 Cursor 的交互开发能力，完成前后端搭建、联调与优化，实现智能数据分析平台的落地。